

Uncertainty-Aware System-Initiated Corrective Feedback for POMDP-Based Robot Planning

Changmin Park, Seongah Park, Nayoung Kim, Sungjin Park, Geonyeong Ko, Junho Lee, and Changjoo Nam

Abstract—In this paper, we present a knowledge-based decision-making framework that enables robots to selectively request human feedback under uncertainty, addressing when to ask and what to ask. We formulate the problem as a partially observable Markov decision process (POMDP) over a STRIPS-style symbolic knowledge base, where the robot maintains multiple world hypotheses and requests state-level feedback only when they cannot be reliably distinguished. Otherwise, it continues autonomous planning and execution. The framework is implemented in a full system architecture integrating sensing, planning, feedback management, and user interaction. We validate the approach in two robotic domains, tomato harvesting and waste sorting, through system experiments and a Wizard-of-Oz user study. Results show that our state-level querying strategy resolves task-relevant ambiguity with fewer interactions than existing policies, while maintaining comparable or higher task success and reducing perceived workload.

Index Terms—Robotic systems, planning under uncertainty, human-robot interaction.

I. INTRODUCTION

Recent advances in robotic systems have improved autonomy in perception, manipulation, and planning, but robots still face perception errors [1], execution failures [2], and unexpected environmental changes [3]. In such environments, human operators can help recover from these failures [4], [5], but continuous monitoring is impractical in long-horizon tasks because it increases workload and fatigue [6]. Robots should therefore recognize uncertain situations during execution and request assistance only when needed [7], [8]. This raises two fundamental questions: *when should a robot ask*, and *what information should it request*?

Recent uncertainty-aware systems selectively request human feedback during task execution [9]–[11]. These approaches can reduce unnecessary intervention and improve decision making under uncertainty. However, uncertainty estimation, query generation, and planning are often treated separately, making it difficult to reason about how feedback will affect downstream execution. The link between when to ask, what to ask, and how the answer should change planning remains insufficiently explored.

To address this challenge, we present a knowledge-based decision-making framework that integrates uncertainty estimation, query selection, and symbolic planning. We formulate

decision making as a partially observable Markov decision process (POMDP) [2] and represent task knowledge using a STRIPS-style symbolic model [12], [13]. During execution, the robot tracks multiple possible world hypotheses and refines them using incoming observations. If the remaining hypotheses cannot be reliably distinguished, the robot requests human feedback to resolve the ambiguity. Otherwise, it proceeds with autonomous planning and execution.

To demonstrate real-world feasibility, the framework is implemented in a robotic system architecture with sensing, planning, feedback management, and user interface modules connected through a centralized knowledge base. Observations and user feedback continuously narrow the set of plausible world hypotheses represented in the planner. This allows the robot to continue autonomous execution and request feedback only if additional information is useful.

We validate the approach in two representative robotic domains (tomato harvesting and waste sorting) through system experiments and a Wizard-of-Oz user study. The system evaluation examines query timing, query content, comparisons with baseline policies, and scalability under increasing task complexity. The user study evaluates task performance, interaction time, workload, and situation awareness to assess the impact of different query strategies on human–robot interaction. Overall, the results show that state-level querying can resolve task-relevant ambiguity with fewer interactions than existing querying policies while maintaining comparable or higher task success. The main contributions of this work are as follows:

- We propose an uncertainty-aware feedback trigger method that decides *when* the robot should ask for help.
- We introduce an information-driven query selection approach to decide *what* the robot should ask to reduce ambiguity among competing world hypotheses.
- We integrate uncertainty estimation, feedback requests, and symbolic planning into a single execution loop.
- We demonstrate that the proposed framework can improve task performance while reducing unnecessary human feedback through system experiments in two representative robotic domains.

II. RELATED WORK

We review related work on structured task knowledge, planning under uncertainty, and uncertainty-driven human feedback. Table I compares representative approaches by their use of interpretable knowledge, symbolic action models, uncertainty-aware planning, human feedback, and system-initiated queries. A circle indicates explicit support, whereas

Changmin Park, Seongah Park, Nayoung Kim, Sungjin Park, and Geonyeong Ko are with Sogang University, Seoul, Republic of Korea.

Junho Lee and Changjoo Nam are with Institution2, Seoul, Republic of Korea.

Corresponding author: Junho Lee and Changjoo Nam (email: cmp9877@gmail.com).

a triangle indicates partial or implicit support without a dedicated symbolic formulation.

A. Knowledge-Based State and Action Models

In robotics, a knowledge base represents task-relevant information in a structured and interpretable form [66], [67]. Prior work has used symbolic predicates [13], [24], temporal logic [15], ontologies [21], and scene graphs [68], [69] to describe objects, attributes, relations, goals, and constraints. This interpretability is important because robots can reason over explicit task facts, while users can inspect, correct, or confirm the same facts during interaction.

However, knowledge alone is not sufficient for robot execution. A robot also needs a transition model that specifies which actions are feasible in a state and how those actions change the state. Symbolic planning provides a representative formulation of such transition models through planning languages such as STRIPS [13], PDDL [24], and FDR [25]. In these representations, states are defined by predicates, and actions are defined by preconditions and effects. This structure allows classical planning methods [26]–[28] to compute action sequences over a well-defined symbolic state space. For example, knowing whether an object is `plastic` or whether a tomato is `ripe` is not merely a semantic label. These facts change which placement, harvesting, or discarding actions are valid. As shown in Table I, classical symbolic planning provides interpretable knowledge and symbolic action models, but it is usually limited in considering partial observability, noisy observations, and uncertain inferred facts.

Knowledge-based robotic systems also use symbolic structure to store observed facts and infer unobserved knowledge from existing information. Logical and relational inference can combine multiple relations and derive plausible hidden states from contextual knowledge [14], [15]. For example, robots can infer the likely location of unseen objects from qualitative spatial relations, container-location relations, or place-object co-occurrence knowledge [16], [17]. Commonsense reasoning further connects symbolic knowledge with perceptual evidence in partially observable environments [18]–[20], while ontologies and type hierarchies support consistent integration and generalization across objects, actions, and task contexts [21]–[23]. These methods help organize uncertain task knowledge, but they usually do not define how uncertainty in a state fact changes action feasibility through an explicit transition model.

Recent work has also used large language models (LLMs) to represent and use task knowledge in more flexible forms, including natural language knowledge and commonsense priors. One line of work uses LLMs directly as planners. These methods generate task decompositions or action sequences and ground them with robot affordances [29], [30], programmatic interfaces [31], [32], or execution feedback [33]. Other methods enrich this process with scene graphs and open-vocabulary spatial representations [34], [35], or with retrieved demonstrations, procedural knowledge, and task constraints [36]–[40]. This line is flexible, but action feasibility is difficult to guarantee if the plan itself is produced by the LLM.

Another line uses LLMs to generate formal task specifications, such as PDDL or temporal logic. The generated

specification is then solved by off-the-shelf planners, automata, or verification mechanisms [41]–[45]. Recent temporal-logic-based systems follow this direction by translating language instructions into LTL or hierarchical temporal-logic specifications [70]–[74]. This improves executability, but it inherits the limitation discussed above: the resulting pipeline still provides limited support for uncertainty in observed or inferred state facts.

Together, these works motivate the axes in Table I: interpretable task knowledge, symbolic transition models, uncertainty-aware planning, and human feedback. Interpretability is essential for feedback because users can only verify or correct uncertainty when the robot exposes it as meaningful task facts. Our work focuses on this intersection by maintaining uncertainty over symbolic task facts while using the transition model to evaluate their effect on feasible actions.

B. Planning under Uncertainty

Robotic uncertainty often stems from partial observability, where the full world state is not directly available. POMDPs provide a standard formulation for planning under such uncertainty. We review POMDP planning, belief approximation, and symbolic extensions.

1) *POMDP*: Unlike deterministic symbolic knowledge-based systems that assume a single world state, POMDPs represent uncertainty as a belief over multiple possible states. In robotic environments, such uncertainty naturally arises from sensor noise, occlusions, incomplete observations, and unexpected state changes. Sequential decision-making under these conditions has been extensively studied within the framework of POMDPs [46]–[49].

A key distinction between a POMDP and an MDP is the introduction of a belief state, which represents a probabilistic distribution over possible states when the true state cannot be directly observed. Accordingly, a wide range of solution methods have been proposed, including point-based value iteration [75], online tree search [76], and Monte Carlo tree search (MCTS) [77].

Despite these advances, effectively representing and updating the belief state remains a fundamental challenge. Since a belief is inherently a continuous distribution over the state space, exact inference is computationally difficult. Moreover, as the state space grows, the belief space expands combinatorially, making inference increasingly intractable. As a result, various approaches have been proposed to approximate the belief state.

2) *Belief Approximation*: Studies on belief approximation can be broadly categorized into particle-based and factored approaches. Particle-based methods approximate the belief as a set of sampled states and have been widely adopted due to their natural compatibility with sampling-based online planning [78], [79]. For example, [77] maintains a set of particles as a history-conditioned belief and estimates action values by sampling future trajectories from a generative model without performing exact Bayesian belief updates. Similar sampling-based ideas have also been extended to large and continuous POMDPs, where DESPOT [46] evaluates a limited

TABLE I: Qualitative comparison of related works. A symbolic action model denotes an explicit task-level model of action preconditions and effects.

Related Works	Interpretable Knowledge	Symbolic Action Model	Uncertainty Planning	Human Feedback	System-Initiated Query
Logic-based knowledge [14]–[23]	○	×	△	×	×
Symbolic planning [13], [24]–[28]	○	○	×	×	×
LLM-based approaches [29]–[45]	△	△	△	×	×
POMDP-based planning [46]–[54]	×	×	○	×	×
Symbolic uncertainty planning [55]–[59]	○	○	○	×	×
Proactive assistance [60]–[65]	×	×	△	○	○
KnowNo-style action queries [9]–[11]	○	×	○	○	○
Ours	○	○	○	○	○

set of sampled scenarios and POMCPOW [80] controls search tree growth in continuous action and observation spaces.

In robotic domains, particle-based belief representations are widely used in localization, navigation, SLAM, object search, and manipulation, where particles encode hypotheses over robot, map, and object states under uncertainty. In navigation, [50] combines POMCP with model predictive control to maximize information gain over occupancy-grid maps, while [51] uses hierarchical local online POMDP planning for exploration in large unknown environments. Similar representations have also been used for continuous control under uncertainty, including autonomous driving at occluded intersections [81] and UAV collision avoidance or target tracking under partial observability [52], [82]. In object search and manipulation, particles often represent complete scene hypotheses. For example, [53] samples visible and occluded object configurations for cluttered object search, while [54], [83] uses particle-based scene hypotheses for multi-object manipulation. Collectively, these studies show that particle beliefs provide a flexible approximation for preserving multimodal dependencies among robot, environment, and object states.

In contrast, factored belief representations model the belief by decomposing the state into a set of variables and exploiting their structure [84]–[86]. Such approaches have been widely used in structured probabilistic models, including dynamic Bayesian networks and Hidden Markov Models [87], where the state is represented in a factored form. To improve computational efficiency, various approximate inference methods have been proposed to reduce the complexity of belief updates by relaxing dependencies among variables. For example, [88] approximates belief updates by maintaining a factorized representation while partially accounting for dependencies between variables. This relaxation can be problematic in symbolic planning, where predicates are coupled by action preconditions, effects, and mutual constraints. Particle-based beliefs instead preserve complete state hypotheses, making them better suited to represent dependencies among symbolic facts during planning.

3) *Information-Gathering Objectives*: Information gathering can also be formulated through value-aware decision objectives. In decision-theoretic formulations, value of infor-

mation, value of perfect information, and value of computation select observations, sensing actions, or computations according to their expected utility improvement rather than uncertainty reduction alone []. Related POMDP variants with belief-dependent rewards, such as ρ -POMDP formulations, can also encode information gain, sensing cost, or query cost directly in the reward objective []. These approaches provide a principled way to align information acquisition with downstream task value.

Our query selection has a different role. It uses expected entropy reduction as a lightweight predicate-level disambiguation criterion within the current symbolic frontier, while long-horizon task optimization is handled by the POMDP planner through task rewards. This distinguishes our setting from reward-optimal information acquisition and active perception methods, where the robot usually chooses sensing or viewpoint actions to improve its own observations []. In our framework, the information action is a user-facing predicate verification grounded in the shared knowledge base.

4) *Symbolic Planning under Uncertainty*: As described in Section II-A, symbolic planning formalisms are interpretable because they describe planning problems through explicit constraints and state transition rules [89], [90]. However, standard predicate-based representations typically assign each fact a binary truth value, making it difficult to directly model observation noise or state uncertainty.

Another line of work addresses uncertainty within symbolic planning through conformant, contingent, conditional, or knowledge-level planning []. These methods extend classical planning by reasoning over incomplete information, sensing actions, conditional branches, or knowledge states so that a plan can depend on observations obtained during execution. Such formulations are closely related to our use of symbolic predicates under partial observability, because they also treat information gathering as part of task execution. However, they typically construct conditional plans or policies over symbolic belief or knowledge states, and the information-gathering actions are usually robot sensing actions rather than human-facing predicate queries. In contrast, our framework performs online POMDP planning and uses the current symbolic frontier to decide which uncertain predicate should be verified by the

user during execution.

To address this limitation, various approaches have attempted to integrate probabilistic reasoning into logical representations. For example, [91] assigns weights to logical rules to enable probabilistic inference over uncertain relationships, and [92] combines logical structure with probabilistic reasoning. However, these approaches primarily focus on building inference models, and their high cost in model construction is unsuitable for online decision-making with complex constraints.

As a result, several studies have explored planning under uncertainty while preserving symbolic structure [55], [56]. For example, [57] encodes commonsense knowledge in ASP [90] to construct a prior over possible worlds and integrates it with probabilistic planning [93]. However, since the belief is defined over a set of possible worlds, the computational cost increases as the number of states grows. To address this, some works organize possible worlds in a structured manner. For instance, [58] construct a state space tree and a hierarchy of POMDPs based on domain knowledge, reducing complexity by leveraging abstraction. More recently, large language models have been used for belief approximation, as in [59], which builds a tree of hypotheses and uses LLMs to generate particles in open-ended settings. While these approaches enable more flexible or scalable belief representations, they either depend on predefined hierarchical structures or rely on the quality and consistency of generated hypotheses.

Overall, prior work addresses the uncertainty-aware planning axis in Table I while showing that symbolic structure can make uncertain planning more interpretable. However, belief maintenance remains costly if many symbolic hypotheses must be considered. This motivates approaches that focus planning on task-relevant reachable hypotheses rather than maintaining a global belief over all possible worlds. It also raises a complementary question tied to the human-feedback and system-initiated-query axes in Table I: when uncertainty remains unresolved, what information should a robot obtain to directly disambiguate competing symbolic states? We next review human feedback and query-based systems that address this question.

C. Human Corrective System

Robots still fail in partially observable environments because uncertainty cannot always be resolved from sensing and planning alone. Human intervention is therefore an important mechanism for correcting errors, resolving ambiguity, and improving robustness [94], [95]. This corresponds to the human-feedback axis in Table I. In human-on-the-loop (HOTL) systems, robots execute tasks autonomously while human operators monitor the system and revise or override decisions when failures occur [96], [97].

However, HOTL systems require operators to sustain attention and detect failures during robot operation. To provide effective corrective feedback, operators must first notice that the robot is in an uncertain or erroneous state. Situation awareness research suggests that operators must then understand the current situation, project possible consequences, and

determine an appropriate intervention [98]. This cognitive process can be difficult in repetitive tasks. Operators may experience vigilance decrement because continuous exposure to monotonous monitoring tasks can slow responses and reduce detection accuracy [99], [100]. Operators may also experience inattention blindness, in which visible failures are missed if attention is not directed to the relevant event [101]. As a result, continuous supervision can increase workload and fatigue [102]. Continuous supervision can also reduce the likelihood that operators detect critical robot failures [102], [103].

Prior work has shown that robots can improve task performance, team efficiency, and subjective satisfaction if they proactively provide assistance during human-robot collaboration [104]. This finding suggests that proactive robot behavior can support collaboration by reducing the need for humans to manage every decision or detect every need for intervention. Related help-seeking systems decide when to request expert actions, manual control, or assistance based on policy uncertainty, task difficulty, or trust dynamics [60]–[63]. Clarification systems similarly ask questions when model uncertainty is high in embodied task completion or collaborative dialogue [64], [65].

Recent uncertainty-aware query systems further study when a robot should initiate interaction. A representative example is [9] (KnowNo), which uses conformal prediction to form a prediction set over candidate robot actions and asks the user when the set is not sufficiently resolved. Related LLM planning systems extend this direction: [10] combines retrieval-augmented reasoning with calibrated plan selection, and [11] uses uncertainty over candidate plans to decide when the robot should ask for help. These methods provide principled mechanisms for system-initiated queries. However, their queries are typically action-level choices, expert interventions, or clarifications among candidate plans. Such queries can require users to infer current state of robot and understand how candidate actions affect future execution. This shifts part of the situation-awareness burden to the user. The user must interpret the current state, project action consequences, and choose an intervention. Our framework instead asks users to verify symbolic variables grounded in the knowledge base, so the requested feedback is stated as a task-level fact rather than an action decision.

This difference in query content changes what users must understand and decide during interaction. Thus, operator burden is also relevant if analyzing system-initiated corrective interaction. Existing studies mainly report task success, uncertainty calibration, and query efficiency. However, workload can affect task performance and intervention quality in automated systems that involve human operators [105]–[107].

Motivated by this consideration, we include operator workload as a user-facing measure alongside task performance. In our interaction structure, the robot monitors its belief uncertainty and asks for corrective feedback if additional information is useful. Users provide targeted input at system-identified moments. This positions our system in Table I as combining human feedback and system-initiated queries with interpretable symbolic knowledge, action constraints, and

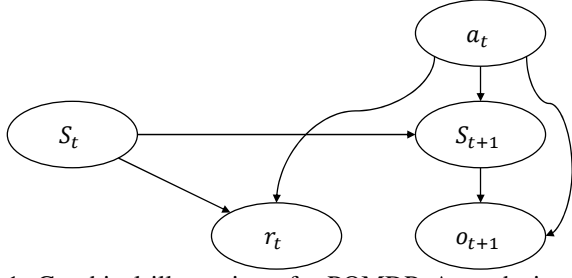


Fig. 1: Graphical illustration of a POMDP. At each time step t , the agent selects an a_t based on its current belief over the latent state s_t . The environment transitions to the next state s_{t+1} and generates a reward r_t . Since the true state is not directly observable, the agent receives an observation o_{t+1} .

planning under uncertainty.

III. PRELIMINARIES

A. Symbolic State and Action Representation

The objective of planning is to find a transition sequence that transforms the initial state into a goal state. The planning problem is defined as a tuple $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, \mathcal{G} \rangle$, where $\mathcal{F}$ is a finite set of atoms, $\mathcal{A}$ is a set of actions, $s_0 \subseteq \mathcal{F}$ is the initial state, and $\mathcal{G} \subseteq \mathcal{F}$ is the goal condition. A state s is represented as a subset of $\mathcal{F}$. Each action $a \in \mathcal{A}$ is defined as a tuple $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$, where $\text{pre}(a), \text{add}(a), \text{del}(a) \subseteq \mathcal{F}$ denote the preconditions, add lists, and delete lists of the action, respectively. An action a is said to be applicable in state s if and only if $\text{pre}(a) \subseteq s$, denoted as $a \llbracket s \rrbracket$. The state transition is defined as $s \xrightarrow{a} s'$, where $s' = (s \setminus \text{del}(a)) \cup \text{add}(a)$.

A plan is a sequence of actions $\pi = (a_1, \dots, a_T)$ such that the goal condition is satisfied after sequentially applying the actions starting from the initial state, i.e.,

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_T} s_T \quad \text{and} \quad \mathcal{G} \subseteq s_T.$$

B. POMDP

A POMDP is defined as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and Ω is the observation space. The transition model $T(s' | s, a)$ specifies the probability of reaching the next state s' after taking action a in current state s . The observation model $O(o | s', a)$ defines the probability of receiving observation o after arriving at state s' by executing action a . The reward function $R(s, a)$ gives a scalar value to each transition, and $\gamma \in (0, 1]$ is the discount factor.

Since the true state is not directly observable, the agent maintains a belief b , which is a probability distribution over $\mathcal{S}$ such that $b(s) = \Pr(s)$. After executing an action a and receiving an observation o , the belief is updated using a Bayes filter.

$$b'(s') = \eta O(o | s', a) \sum_{s \in \mathcal{S}} P(s' | s, a) b(s), \quad (1)$$

where η is a normalization constant.

A policy π maps beliefs to actions. The objective of a POMDP is to find a policy that maximizes the expected cumulative discounted reward, defined as $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$.

C. POMCP

Partially Observable Monte Carlo Planning (POMCP) performs Monte Carlo tree search over action-observation histories $h = \langle a_1 o_1 \dots a_t o_t \rangle$, where each node in the search tree corresponds to a history. Action selection follows the UCB rule:

$$a^* = \arg \max_a \left(V(ha) + c \sqrt{\frac{\log N(h)}{N(ha)}} \right), \quad (2)$$

where $V(ha)$ is the estimated value of a after h , $N(h)$ and $N(ha)$ denote visit counts, and c is an exploration constant.

Each simulation samples the next state, observation, and reward from a generative model $(s', o, r) \sim \mathcal{G}(s, a)$. The return is computed recursively as $R = r + \gamma R'$, where R' is the return from subsequent steps. The value estimate is updated using an incremental average:

$$V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}. \quad (3)$$

The belief associated with a history h is represented implicitly as a set of particles, $B(h) = \{s \mid s \text{ is visited at } h\}$, which approximates the posterior over states without explicit belief updates.

IV. SYSTEM ARCHITECTURE AND METHODS

In this section, we present an overview of the system architecture and the data flows between modules. Then we give a detailed explanation of their functionalities and applications.

A. Overview

Fig. 2 illustrates the brief system architecture and the data flow between the main modules. We integrate the *Knowledge Base* (KB), the *Planning Manager* (PM), the *Sensing and Perception* module (SP), the *Feedback Manager* (FM), and the *User Interface* (UI). The KB maintains symbolic descriptions and facts used for planning, SP generates sensory information from robot sensors, PM generates and executes plans, FM coordinates corrective feedback, and UI bridges the system and the user.

The KB stores essential information required for the robot to perform tasks and explicitly represents the current state of the system. All modules share information and interact with each other through the KB. The KB is composed of three types of information: domain knowledge, static information, and local information. Domain knowledge refers to prior knowledge in a specific domain. It includes descriptions of robot skills, domain-specific rules, and general explanations of the environment. Static information is used during planning and execution but remains unchanged throughout task execution. Unlike domain knowledge, static information is encoded in a structured form and includes information such as robot

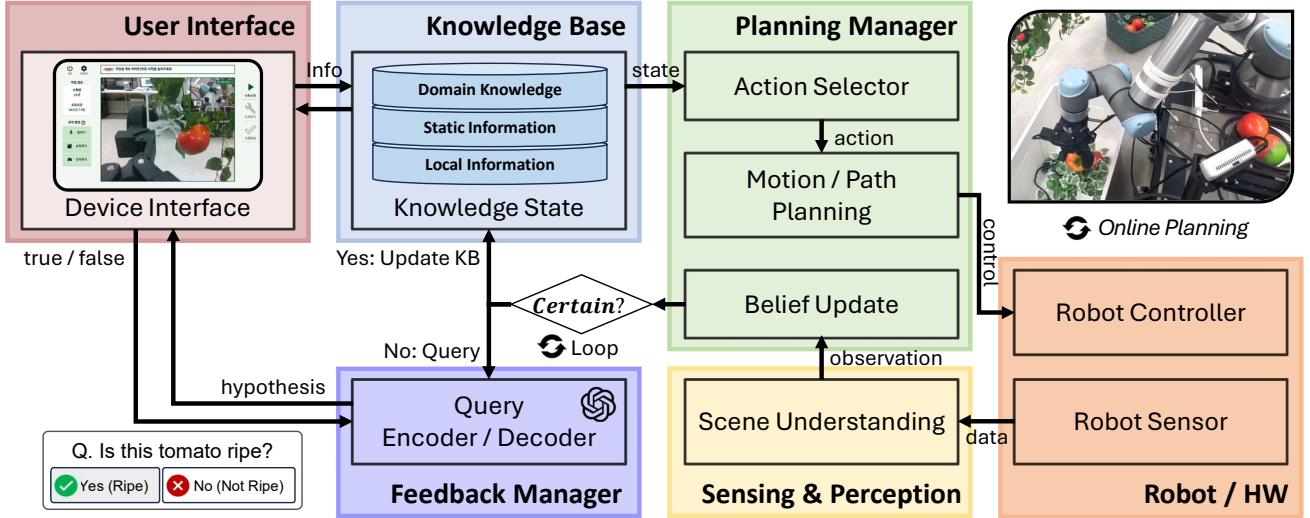


Fig. 2: System architecture and data flow of the proposed framework. The system consists of the *Knowledge Base* (KB), *Planning Manager* (PM), *Sensing and Perception* module (SP), *Feedback Manager* (FM), and *User Interface* (UI). The PM performs online planning based on the current knowledge state stored in the KB and updates the belief using observations from SP. The system evaluates the confidence of the updated belief. If the confidence exceeds a threshold, the most probable hypothesis is committed to the KB. Otherwise, the FM generates a query and requests corrective feedback from the user through the UI. The feedback is then used to refine the belief and continue planning.

parameters and local maps. In contrast, local information is used during planning and execution but cannot be known in advance. It is obtained through observations from the SP module and stored in the KB in the form of facts.

The PM is responsible for online action selection and execution under partial observability. The POMCP-based planner within the PM module selects the best action from the current belief and triggers the SP module to acquire additional observations. For instance, if the robot is positioned in front of a tomato stem, it naturally executes a *detect* action to infer the locations and ripeness states of tomatoes. To quantify uncertainty, the PM approximates the belief in the form of particle and computes the entropy over the resulting belief distribution. If such uncertainty is detected, the PM queries the FM to obtain additional information that can resolve the ambiguity of the observation. Upon receiving a response from the FM, the PM updates the belief accordingly and promotes the most probable belief state to a certain state in the knowledge base.

The SP module provides the current visual sensing information to the UI in real time. In addition, it updates the KB with observations triggered by the PM. In this process, the SP module makes use of data from various sensors, such as cameras, LiDAR, and tactile sensors, to extract information used for task execution, including object locations, states, and properties. For example, from RGBD images, the SP module infers visual attributes such as the position, ripeness, and state of a tomato, while tactile sensors are used to perceive contact-based properties such as object stiffness or softness. The high-level perceptual information extracted by the SP module is stored in the KB in the form of symbolic facts.

The FM module converts symbolic hypotheses from the PM into user-facing questions using a predefined predicate-

to-template mapping. For example, a hypothesis such as $\text{ripe}(\text{tomato}_1)$ is mapped to a question such as “Is tomato 1 ripe?”. Along with the query, the UI highlights the referenced object with a bounding box. The FM then maps the response from the user to a boolean value, which is used by the PM to filter the belief.

Finally, the UI displays the current visual sensing information and the system state, and allows the user to provide direct feedback or make selections. The user interacts with the FM module through the UI to monitor and supervise task execution.

1) *Knowledge Base*: The KB maintains a structured representation of the environment, including objects, properties, task-relevant relations, numeric quantities, and their associated confidence values. Formally, we define $\mathcal{KB} = \mathcal{I}^D \sqcup \mathcal{I}^S \sqcup \mathcal{I}^L$,¹ where $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ denote the domain, static, and local information sets, respectively, as described in Sec. IV-A.

For $\mathcal{I}^S$ and $\mathcal{I}^L$, each information set contains both symbolic facts and numeric fluents. A symbolic fact is defined as $f = \text{pred}(\mathbf{x})$, where $\text{pred}(\cdot)$ denotes a predicate symbol and $\mathbf{x}$ denotes its arguments. A numeric fluent is defined as $\nu = \langle g(\mathbf{x}), v \rangle$, where $g(\mathbf{x})$ denotes a numeric function over objects and $v \in \mathbb{R}$ denotes its value. For example, symbolic facts can represent object categories and relations, such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, while numeric fluents can represent measurable quantities, such as $\text{pose_x}(\text{tomato}_1) = 0.12$ or $\text{distance}(\text{robot}, \text{stem}_1) = 2.5 \text{ (m)}$.

Specifically, the set $\mathcal{I}^S$ contains information that remains invariant throughout planning. By definition, for all $a \in \mathcal{A}$, $\text{Effect}(a) \cap \mathcal{I}^S = \emptyset$, where $\text{Effect}(a) = \text{add}(a) \cup \text{del}(a)$ denotes symbolic effects and can be extended to include numeric

¹The notation $\sqcup$ denotes a disjoint union, indicating that $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ form a partition of $\mathcal{KB}$.

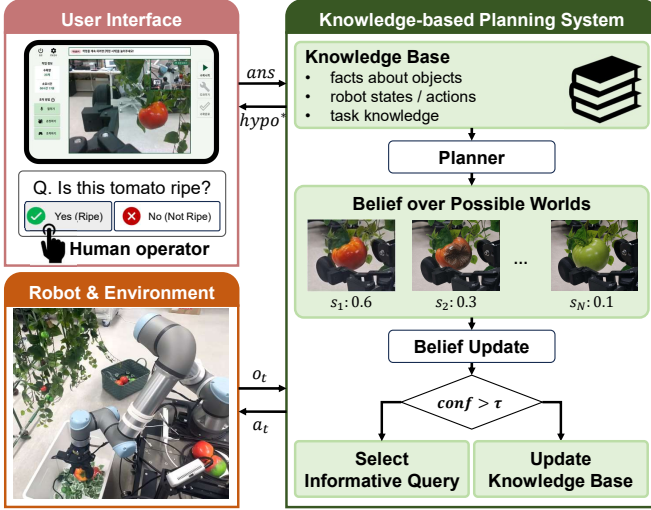


Fig. 3: Planning Manager workflow. The PM maintains a belief over symbolic possible worlds, updates it using observations, and evaluates belief uncertainty. If uncertainty exceeds a threshold, the PM requests verification through the FM. Otherwise, it commits the most probable hypothesis to the KB and continues planning and execution.

updates. These elements capture structural properties of the environment that are not altered by any action. For example, predicates such as `tomato(tomato1)` and `loc(stem1)`, as well as fixed numeric quantities like `loc_x(stem1) = 1.2`, remain unchanged regardless of the executed actions.

In contrast, $\mathcal{I}^L$ denotes local information obtained through the SP module and is not known a priori. This includes perceptual predicates, such as object existence or category estimates, as well as numeric fluents estimated from sensor data. For example, predicates such as `ripe(tomato1)` or `detected(tomato2)` may be inferred from observations, and numeric values such as `distance(robot, stem1) = 2.3` or `pose_x(tomato1) = -0.5` are updated as new sensor measurements become available.

2) *Planning Manager*: The PM performs decision making based on the current belief state while continuously updating the knowledge base during execution, as illustrated in Fig. 3. Uncertainty is quantified using the entropy of the belief distribution. If the uncertainty exceeds a threshold, the planner requests additional verification from the FM. Otherwise, it commits the most probable hypothesis to the knowledge base and proceeds with action execution. In the following, we describe the belief approximation, the uncertainty estimation, the POMCP based search, and the overall planning procedure.

a) *Belief representation*: In partially observable environments, the true state cannot be directly observed, and thus the PM maintains a belief, which is a probability distribution over states. The belief at time t is defined as $b_t(s) = P(s_t = s \mid h_t)$, where $h_t = \{a_1, o_1, \dots, a_t, o_t\}$ denotes the history of actions and observations. However, in symbolic domains, the state space is large and subject to structural constraints, making exact belief tracking computationally intractable. Thus, we approximate the belief using a set of particles.

Formally, at time step t , the belief is represented as a finite

set of particles and their associated weights:

$$\mathcal{B}_t = \{(s_i, w_i)\}_{i=1}^N, \quad \sum_{i=1}^N w_i = 1,$$

where each s_i denotes a possible world composed of a finite set of facts. The approximated belief distribution is then

$$b_t(s) \approx \sum_{i=1}^N w_i \delta(s_i = s).$$

In the symbolic domain, actions exhibit partially predictable effects due to their interpretable structure. For example, `pick(tomato1)` may result in grasp failure or selecting a different tomato while leaving the robot location unchanged. Based on this observation, we define a finite set of distinct reachable successor states, referred to as the *frontier*. For each s and a , the frontier is defined as

$$Frontier(s, a) = \{s_1^*, s_2^*, \dots, s_j^*\},$$

where each s_k^* denotes a reachable successor state and $s_k^* \neq s_\ell^*$ for $k \neq \ell$.

The frontier is generated from the symbolic transition model $T(s' \mid s, a)$ by applying action a to the current state. We use the STRIPS-style action representation introduced in Sec. III-A, and action selection is restricted to actions applicable in the sampled symbolic state. Deterministic effects are applied directly. For uncertain perceptual or execution effects, the action model enumerates the possible symbolic successors with nonzero transition probability. Thus, the frontier is the finite support of $T(\cdot \mid s, a)$ under the symbolic transition model, and the prior frontier weight is given by $w_k = T(s_k^* \mid s, a)$. For example, an applicable `pick(tomato1)` action may produce a success successor with probability 0.8, in which the symbolic effect of picking is added to the state, and a failure successor with probability 0.2, in which the robot remains in a hand-empty state. These successors become elements of $Frontier(s, a)$ with prior weights 0.8 and 0.2, respectively.

In contrast to standard particle filtering, we assume a *knowledge* state s^K that represents the set of certain facts. Given a and s^K , we construct the belief by considering only the reachable states induced by s^K . That is, particles are defined exclusively over the frontier $Frontier(s^K, a)$, and the belief is represented as

$$\mathcal{B}_t = \{(s_k^*, w_k)\}_{k=1}^j, \quad \sum_{k=1}^j w_k = 1,$$

where each $s_k^* \in Frontier(s^K, a)$ and $w_k = T(s_k^* \mid s^K, a)$ before incorporating the new observation. The belief is therefore maintained over the currently reachable frontier rather than over all symbolic worlds. After executing a and receiving an observation o , the belief over the frontier is updated by adjusting the weight of each particle according to how well it explains the observation. Specifically,

$$\forall s_k^* \in Frontier(s^K, a), \quad w'_k = w_k \cdot O(o \mid s_k^*, a). \quad (4)$$

Here, $O(o \mid s_k^*, a)$ is the action-conditioned observation

likelihood defined by the SP module. It measures how likely the received observation o would be if the frontier state s_k^* were the true successor state after action a . In the implementation, this likelihood is computed from the domain-specific observation model: for example, waste sorting uses the object-observed and category-classification probabilities for `detect` observations, while tomato harvesting uses the detection and scan observation probabilities for tomato state observations. The probabilities used in the reported experiments are summarized in Appendix B-B.

The complete belief-update operator used in Alg. 1 consists of frontier generation, likelihood weighting by Eq. 4, normalization of the particle weights, and removal of zero-likelihood particles. The normalized belief is then used to compute the *uncertainty* of the current state.

b) Uncertainty of the state: Given the updated belief over the frontier, uncertainty is quantified based on the distribution of particle weights. Intuitively, uncertainty increases if multiple candidate states have comparable likelihoods, and decreases if the belief is concentrated on a single dominant state. This uncertainty arises from *hypotheses*, which denote literals whose truth values are not yet determined.

The confidence of the current belief is defined as

$$\text{Conf}(\mathcal{B}) = 1 - \frac{\mathcal{H}(\mathcal{B})}{\log_2 j}, \quad (5)$$

where j and $\mathcal{H}(\mathcal{B})$ denote the number of particles and the entropy of the particle weights, respectively. In the degenerate case where $j = 1$, the belief is fully concentrated on a single state and we define $\text{Conf}(\mathcal{B}) = 1$. Let $\tilde{w}'_k$ denote the normalized weights such that $\tilde{w}'_k = w'_k / \sum_{l=1}^j w'_l$. The entropy is computed as

$$\mathcal{H}(\mathcal{B}) = - \sum_{k=1}^j \tilde{w}'_k \log_2 \tilde{w}'_k.$$

This normalization ensures that the confidence lies in $[0, 1]$, where a uniform belief yields low confidence and a concentrated belief yields high confidence.

If the confidence exceeds a predefined threshold τ , the most probable state is selected as the updated knowledge, $s^* = \arg \max_{s_k^*} \tilde{w}'_k$, and the belief collapses to this committed knowledge state. The next planning step then constructs a new frontier from the committed knowledge state and the selected action. Otherwise, the system identifies uncertainty-inducing hypotheses that differentiate candidate states.

The candidate hypotheses are constructed from literals that differ between the current knowledge state and the frontier states. Let $\mathcal{C}$ denote this set of candidate hypotheses. For each $c \in \mathcal{C}$, the belief is partitioned into two subsets depending on whether c holds:

$$\mathcal{B}_c^+ = \{(s_k^*, \tilde{w}'_k) \mid c \in s_k^*\}, \quad \mathcal{B}_c^- = \{(s_k^*, \tilde{w}'_k) \mid c \notin s_k^*\}.$$

The system selects the hypothesis that is expected to reduce the uncertainty the most by minimizing the expected entropy after querying:

$$c^* = \arg \min_{c \in \mathcal{C}} [P(c)\mathcal{H}(\mathcal{B}_c^+) + P(\neg c)\mathcal{H}(\mathcal{B}_c^-)], \quad (6)$$

where

$$P(c) = \sum_{s_k^*: c \in s_k^*} \tilde{w}'_k \quad \text{and} \quad P(\neg c) = \sum_{s_k^*: c \notin s_k^*} \tilde{w}'_k.$$

This criterion selects the predicate-level hypothesis that is most informative for the current frontier. For example, the selected hypothesis may be a fact such as $c^* = \text{ripe}(\text{tomato}_1)$ or $c^* = \text{plastic}(\text{waste}_2)$.

The selected hypothesis c^* is passed to the FM as a query. After receiving the answer, the belief is filtered by removing particles that contradict the feedback. If the answer confirms c^* , only particles containing c^* are retained. Otherwise, particles containing c^* are discarded:

$$\mathcal{B} \leftarrow \begin{cases} \{(s_k^*, \tilde{w}'_k) \mid c^* \in s_k^*\}, & \text{if } \text{ans} = \text{true}, \\ \{(s_k^*, \tilde{w}'_k) \mid c^* \notin s_k^*\}, & \text{if } \text{ans} = \text{false}. \end{cases} \quad (7)$$

The remaining weights are renormalized, and the confidence is recomputed. This process is repeated until the confidence exceeds τ or no informative hypothesis remains.

Each particle represents a possible world in the current frontier. Through this process, the user verifies the truth value of one queried predicate fact, allowing the PM to reduce feasible worlds that are inconsistent with the answer. The following corollary states a simple sufficient condition under which this repeated filtering identifies the correct world.

Corollary 1 (Correct identification under complete frontier coverage). Assume that the correct world is contained in the set of possible worlds generated at the frontier. If the user always provides feedback consistent with the correct world, and each feedback step eliminates at least one incorrect candidate whenever ambiguity remains, then the PM eventually identifies the correct world.

Proof. Let $\mathcal{B}$ denote the current finite set of candidate worlds and let $s^\dagger \in \mathcal{B}$ denote the correct world. Since user feedback is assumed to be consistent with $s^\dagger$, the filtering step never removes $s^\dagger$ from $\mathcal{B}$. Whenever more than one candidate remains, the feedback update removes at least one incorrect candidate by assumption. Therefore, because $\mathcal{B}$ is finite, repeated feedback updates eventually remove all incorrect candidates and leave only $s^\dagger$. $\square$

In our framework, this elimination condition is instantiated through predicate facts. Each hypothesis c corresponds to the truth value of a predicate fact with its arguments, and the candidate belief is partitioned into $\mathcal{B}_c^+$ and $\mathcal{B}_c^-$ as above. Correct user feedback selects the subset whose truth value matches the correct world and discards the opposite subset. Thus, when the queried predicate fact separates the correct world from at least one incorrect candidate, the feedback step satisfies the elimination condition in Corollary 1.

c) POMCP-based search: We employ POMCP [77] as the underlying planner for action selection. Our goal is not to optimize the planner itself, but to integrate an off-the-shelf sampling-based planner with symbolic belief representation and human-verifiable uncertainty resolution. POMCP is suitable for this role because its history-tree search samples successor states from a generative model, while our symbolic transition model restricts these samples to applicable actions

and reachable successor states. The detailed pseudocode for the search, simulation, and action-selection routines is provided in Alg. 2 to Alg. 4 in Appendix A.

The search phase follows standard POMCP by repeatedly sampling a state and performing simulations. Each sampled state is passed to `SIMULATE` (line 11), and the accumulated values are used to select the final action. If the belief is empty, the search uses s^K , and at the root, the action is chosen based on value without UCB (line 13).

In the simulation phase, the recursive structure and return computation remain the same as in standard POMCP, but both action selection and node expansion are restricted to $\mathcal{A}(s)$ (line 10). That is, instead of considering the full action space, only actions applicable in state s are explored. If a new history h is encountered, the node is expanded only with actions in $\mathcal{A}(s)$ (line 11), and the value is initialized via a `ROLLOUT`. Here, T denotes the search tree, while $N(ha)$ and $V(ha)$ represent the visit count and value of action a , respectively. The next state, observation, and reward are then sampled from the generative model $(s', o, r) \sim \mathcal{G}(s, a)$ (line 15). As a result, the simulation remains stochastic but is confined to reachable symbolic transitions.

The same principle is applied to both action selection and rollout. The candidate action set $\mathcal{A}_{cand} = \{a \mid a[s] \in \mathcal{A}, ha \in T\}$ includes actions that are both applicable in state s and present in the tree. If `use_ucb` is true, an action is selected according to the UCB criterion defined in Eq. 2. Otherwise, the action with the highest value is chosen. During rollout, actions are sampled uniformly from all applicable actions. The generative model $\mathcal{G}(s, a)$ then produces the next state, observation, and reward, while the action space is restricted by symbolic constraints.

In symbolic domains, action preconditions and effects are explicitly defined, so exploring infeasible actions is unnecessary. Restricting the action space to $\mathcal{A}(s)$ therefore improves both search efficiency and model consistency. Overall, the structure follows POMCP while naturally incorporating the constraints imposed by the symbolic representation.

d) Action execution: Each $a \in \mathcal{A}$ selected by the POMCP is associated with an executable robot skill. The symbolic action specifies the task-level intention and parameters, while the corresponding skill defines the motion or perception routine required for execution. For example, a symbolic action such as `navigate(robot, l1, l2)` is mapped to a navigation routine, and `pick(robot, tomato1)` is mapped to a predefined manipulation trajectory.

In this work, $\forall a \in \mathcal{A}$ follows an encoded execution policy that maps a symbolic operator to a robot skill through an off-the-shelf motion planner, such as MoveIt [108]. The planner takes symbolic facts and numeric fluents stored in the KB as inputs. Numeric fluents, such as object poses, distances, and grasping parameters, define the parameters of motion generation and maintain consistency between high-level planning and low-level execution.

Specifically, after each action execution, the system receives an observation o from the environment and uses it for belief update. The system uses these observations to update both symbolic predicates and numeric fluents in the KB. This

Algorithm 1: Planning Pipeline

```

1 Input: initial knowledge  $s_0^K$ , confidence threshold  $\tau$ 
2 Output: plan  $\pi$ 
3 procedure PLANNING ( $s_0^K$ )
4    $knowledge(\mathcal{B}) \leftarrow s_0^K$ 
5    $t \leftarrow 0$ 
6   repeat
7      $a_t \leftarrow \text{SEARCH}(\mathcal{B})$ 
8     if  $a_t = \emptyset$  then
9       // There is no  $a[s]$ . (Dead-end)
10      break
11     execute  $a_t$  and receive observation  $o_{t+1}$ 
12     // UPDATE performs frontier
13     // weighting and normalization
14      $\mathcal{B} \leftarrow \text{UPDATE}(\mathcal{B}, a_t, o_{t+1})$ 
15      $\rho \leftarrow \text{CONF}(\mathcal{B})$  (Eq. 5)
16     while  $\rho \leq \tau$  do
17        $hypo^* \leftarrow \text{SELECT-HYPO}(\mathcal{B})$  (Eq. 6)
18        $ans \leftarrow \text{QUERY-FM}(hypo^*)$ 
19        $\mathcal{B} \leftarrow \text{FILTER}(\mathcal{B}, hypo^*, ans)$  (Eq. 7)
20        $\rho \leftarrow \text{CONF}(\mathcal{B})$ 
21        $s^K \leftarrow \arg \max_{(s_k^*, \tilde{w}_k') \in \mathcal{B}} \tilde{w}_k'$ 
22        $knowledge(\mathcal{B}) \leftarrow s^K$ 
23        $t \leftarrow t + 1$ 
24   until  $t = \text{MAX-STEP}$ ;
25   return  $\pi = (a_0, \dots)$ 

```

design separates task-level decision making from low-level motion planning. The KB provides a shared representation across these levels. As a result, the PM does not depend on a specific motion planner implementation.

e) Overall planning pipeline: We adopt an online planning framework as described in Algorithm 1. The planning process starts from the initial knowledge state s_0^K , which is used to initialize the belief (line 4). At each iteration, the planner first selects a_t by performing Monte Carlo tree search over the current belief $\mathcal{B}$ (line 7). If no applicable action exists, the process terminates. Otherwise, the selected action is executed and an observation o_{t+1} is received from the environment.

The belief is then updated using the observation model (line 11), and the confidence of the updated belief is computed (line 12). If the confidence is below the threshold τ , the planner queries the FM to reduce uncertainty. Specifically, it selects the most informative hypothesis, sends it to the FM, and filters out particles inconsistent with the answer (lines 14–16). This process is repeated until the confidence exceeds τ . Once the belief becomes sufficiently confident, the most probable state is committed as the updated knowledge state, and the planner proceeds to the next step.

3) Sensing and Perception: The SP module acquires raw sensory data, generates local information in $\mathcal{I}^L$, and updates the numeric fluents in the KB. It also provides observations required for belief updates during action execution. Let $\mathcal{M}$ denote the set of perception models. For each predicate

pred associated with local information in $\mathcal{I}^L$, there exists a corresponding perception model $m_{\text{pred}} \in \mathcal{M}$ defined as

$$m_{\text{pred}} : \mathcal{R} \rightarrow \mathcal{I}^L,$$

where $\mathcal{R}$ denotes the space of raw sensory data. Given a sensory input $r \in \mathcal{R}$, the model $m_{\text{pred}}(r)$ produces a symbolic fact $f = \text{pred}(\mathbf{x})$ or a numeric fluent $\nu = \langle g(\mathbf{x}), v \rangle$ in $\mathcal{I}^L$.

Through this process, perceptual predicates such as `ripe(tomato1)` or `detected(tomato2)` are instantiated, and numeric fluents such as $\nu = \langle \text{distance}(\text{robot}, \text{stem}_1), 2.3 \rangle$ or $\nu = \langle \text{pose_x}(\text{tomato}_1), -0.5 \rangle$ are continuously updated based on incoming observations. Since $\mathcal{I}^L$ represents information that is not known a priori, all elements in this set are generated and refined exclusively through the SP module.

In addition, the SP module defines the observation interface of the system. After executing an action $a \in \mathcal{A}$, the system receives an observation $o \in \mathcal{O}$ derived from the current sensory input, which is used to update the belief over possible states. Furthermore, if triggered by the PM, the SP module acquires new sensory input and evaluates the relevant perception models to produce observations and update the associated elements in $\mathcal{I}^L$.

4) *Feedback Manager*: The FM presents hypotheses selected by the PM and converts user responses into a form that can be processed by the system. Each queried hypothesis is represented internally as a predicate, such as `ripe(tomato1)`. The FM maps this predicate to a predefined question template, such as “Is this tomato ripe?”. To support user understanding, the queried object is visualized with a bounding box indicating its location.

Users can respond via touch or speech, and the FM converts these responses into a boolean form (true or false) that can be used for belief updates. This design keeps the feedback interface simple: the planner selects what predicate to verify, while the FM only formats the selected predicate and records the user’s answer.

5) *User Interface*: As shown in Fig. 4, the UI enables interaction between the user and the system and is implemented on a tablet. It presents queries generated by the FM and overlays the queried object on the camera image using a bounding box to support intuitive understanding. Users provide responses through simple touch input, with optional speech input. The interface hides internal representations and is designed to elicit accurate feedback with minimal workload.

V. EXPERIMENTS

Our experiments evaluate the proposed system from two complementary perspectives, system-level performance and human-user interaction. The system evaluation uses an oracle user to isolate the behavior of the planning and query framework under controlled feedback. We measure task success, the number of queries, query rate, and planning length to examine the trade-off between task performance and interaction cost. The user study examines how the resulting query interaction affects participant performance, workload, and situation awareness in practical task settings.

A. Experimental Setup

We evaluate the framework in two task domains, waste sorting and tomato harvesting, using controlled system-level environments for repeated trials. The waste sorting domain is implemented in NVIDIA Isaac Sim with a tabletop scene and a fixed UR5 robotic arm for pick-and-place tasks. The tomato harvesting domain follows the task structure and sensing setup of our real robot platform, which consists of a Summit mobile base, a UR5e arm, an RGB-D camera, and a LiDAR sensor. The repeated system-level evaluations are conducted in a controlled environment to support randomized trials and fair comparisons across conditions. Both domains use YOLOv12 for object detection [109].

1) *Waste sorting domain*: As shown in Fig. 5a, a fixed UR5 robot is positioned between the objects and the recycling bins and performs three actions: `detect`, `pick`, and `place`. We assume that all objects are reachable by the manipulator.

Each object belongs to one of four categories: `general`, `paper`, `can`, and `plastic`. The object categories are not known a priori and must be inferred through perception. Since objects may be partially occluded by others, a single observation may not reveal all categories, requiring repeated detection during execution. The `detect` action can observe multiple objects simultaneously, after which the robot places each object into the corresponding recycling bin.

The initial arrangement of objects is randomized in every episode. A task is considered successful when all objects are correctly sorted within the maximum step limit. A reward of +5 is assigned when a waste object is placed into the recycling bin corresponding to its category, and an additional reward of +10 is given when all objects are correctly sorted. All other actions receive zero reward.

2) *Tomato harvesting domain*: We consider a tomato harvesting scenario in which four tomatoes are attached to two stems, denoted as `stem_1` and `stem_2`, as shown in Fig. 5b. The robot navigates between three locations: `dock_station`, `stem_1`, and `stem_2`, and performs `detect`, `pick`, `scan`, `place`, and `discard` actions.

Each tomato can be in one of three states: `ripe`, `unripe`, or `rotten`. The tomato states are initially unknown and must be estimated through perception. During detection, ripe and rotten tomatoes are visually ambiguous and cannot always be reliably distinguished. To resolve this ambiguity, the robot performs a `scan` action after picking a tomato to verify its condition before harvesting or discarding it. Unripe tomatoes are left untouched.

The tomato locations are randomized across stems in every episode. A task is considered successful when all ripe tomatoes are harvested and all rotten tomatoes are discarded within the maximum step limit. A reward of +5 is assigned when a ripe tomato is correctly harvested or a rotten tomato is correctly discarded after scanning. An additional reward of +10 is given when all task-relevant tomatoes are processed correctly. All other actions receive zero reward.

3) *Implementation Details*: In all experiments, the planner used a discount factor $\gamma = 0.95$, UCB exploration constant $c = 1.0$, maximum simulation depth $\text{depth}_{\text{max}} = 20$, and $\text{TIMEOUT} = 100$ Monte Carlo simulations per planning

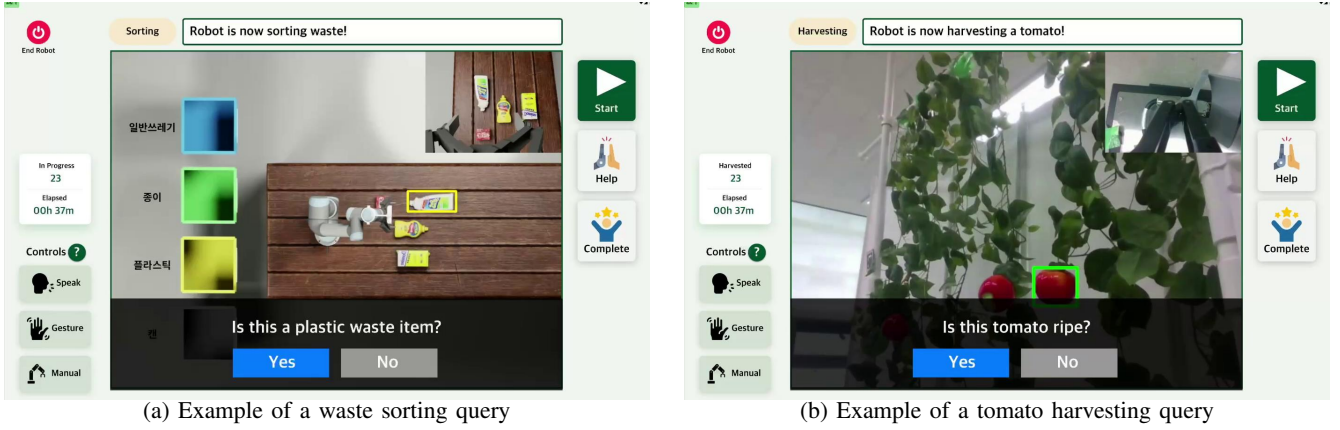


Fig. 4: Examples of the query interface used for human feedback. The interface visualizes the current observations of robot and highlights objects relevant to the queried state. Users provide binary feedback through the touchscreen interface to clarify task-relevant uncertainty. (a) The user identifies the category of a detected waste item in the waste sorting domain. (b) The user confirms the ripeness of a detected tomato in the tomato harvesting domain.

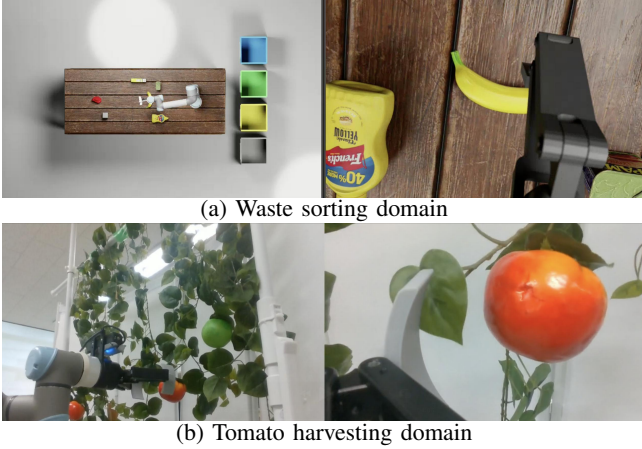


Fig. 5: The illustration of the environments. (a) A UR5 robot sorts four types of waste into designated bins in Isaac Sim. (b) A mobile manipulator harvests ripe tomatoes and discards rotten ones in a real-world setting.

step. Each standard episode was limited to 50 steps. Each reported threshold or policy result aggregates five scenario configurations with 40 randomized episodes per scenario, yielding 200 episodes per domain-condition cell. Randomized episodes are generated from logged pseudorandom seeds. Detailed implementation settings are provided in Appendix B.

B. System Evaluation

We evaluate the proposed framework from three perspectives. We first examine how the confidence threshold τ influences the trade-off between task performance and user feedback. We then compare the proposed query strategy with representative baselines under identical task settings. Finally, we analyze the computational scalability of the method as the symbolic state space increases.

1) *Threshold Ablation:* We first evaluate the effect of the confidence threshold τ on system performance. Fig. 6 and

Table II report the resulting success rate, execution steps, and number of user queries across different threshold values.

The results show that the two domains respond differently to the confidence threshold. In waste sorting, performance improves rapidly once a small amount of uncertainty is resolved, increasing from 11.5% at $\tau = 0.1$ to 82.0% at $\tau = 0.2$. Because task success primarily depends on identifying the correct waste category, additional feedback provides limited benefit once the category is determined. In contrast, tomato harvesting involves a longer sequence of dependent decisions, including navigation, detection, picking, scanning, and execution. Unresolved uncertainty can therefore propagate through multiple stages of the task, causing performance to remain low at small threshold values. Success increases substantially only at higher thresholds and reaches 82.5% at $\tau = 0.8$.

Increasing τ consistently increases interaction because the planner requests feedback for a larger fraction of uncertain beliefs. This trend is reflected in both the average number of queries and the query rate. However, the additional interaction does not always translate into proportional gains in task success. In tomato harvesting, increasing the threshold from $\tau = 0.8$ to $\tau = 1.0$ increases the average number of queries from 29.4 to 48.3 and the query rate from 0.49 to 1.00, while success changes only slightly from 82.5% to 84.0%. This suggests that over-querying can introduce unnecessary interaction without substantially improving downstream decisions. In waste sorting, by contrast, the simpler category-assignment structure benefits from high confidence and reaches 95.5% at $\tau = 1.0$. Based on the cross-domain trade-off between success and interaction cost, we use $\tau = 0.8$ in the remaining experiments.

2) *Baseline Comparison:* We next compare the proposed query strategy with representative alternatives. **All** requests feedback whenever multiple frontier hypotheses remain, regardless of the current confidence level. **No** performs autonomous execution without user feedback. **Random** selects queries uniformly at random. **KnowNo** [9] is a GPT-4o-based action-query baseline [110]. In contrast, **Ours** selectively

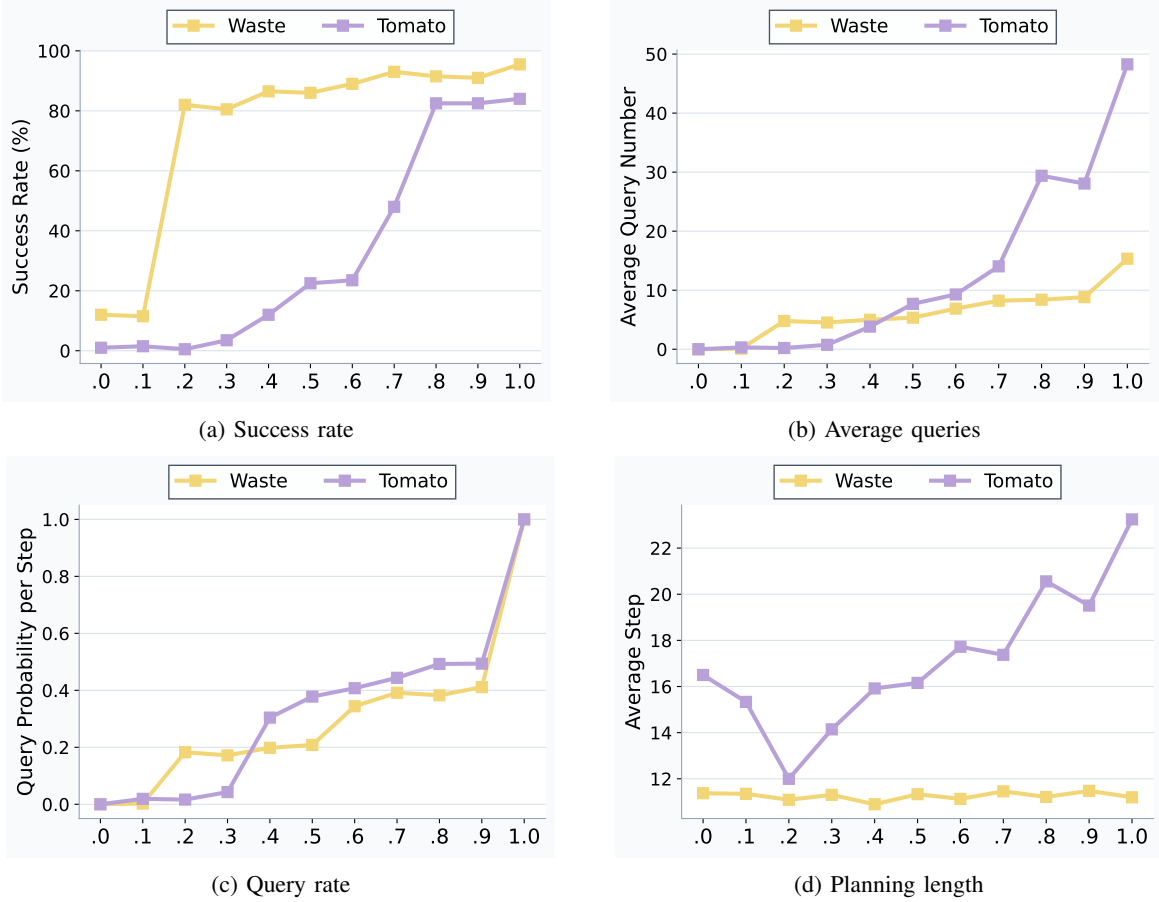


Fig. 6: Effect of the confidence threshold τ on system behavior. (a) Task success rate. (b) Average number of questions per episode. (c) Query probability per execution step. (d) Planning length for successful runs. Increasing τ generally increases feedback, but very high thresholds can add interaction without improving success.

requests state-level feedback based on belief confidence and expected information gain. All methods are evaluated under identical task settings using $\tau = 0.8$.

We note that **All** is not equivalent to setting $\tau = 1.0$. The **All** policy requests feedback whenever multiple frontier hypotheses remain, whereas $\tau = 1.0$ continues querying until complete confidence is achieved. Consequently, **All** may terminate querying even when uncertainty remains, while $\tau = 1.0$ attempts to eliminate all uncertainty before acting.

All achieves high success rates in both domains. In waste sorting, it obtains the highest success rate of 97.5%. In tomato harvesting, it achieves a success rate of 81.5%. These gains come at the cost of frequent interaction, requiring an average of 15.9 and 47.3 queries in the two domains, respectively. The query rate reaches 1.00 in both domains because feedback is requested whenever a query is available. Despite this intensive supervision, the average number of execution steps remains comparable to the other feedback-based methods, suggesting that additional feedback mainly improves decision quality rather than shortening the task itself.

As expected, **No** performs poorly in both domains, achieving only 13.5% success in waste sorting and 1.5% in tomato harvesting. Without user feedback, incorrect observations remain uncorrected and propagate through subsequent symbolic

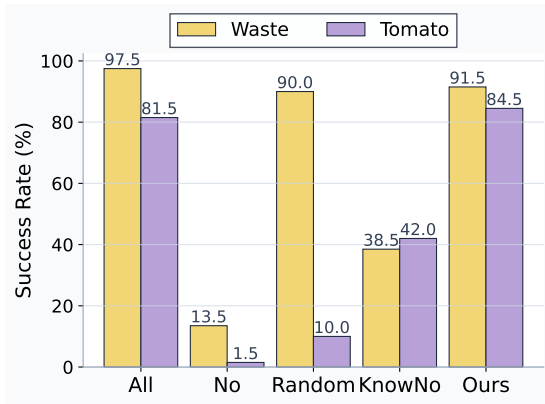
decisions. **Random** improves over **No** in waste sorting, increasing the success rate from 13.5% to 90.0%. This relatively strong waste-sorting result reflects the shorter task horizon and the fact that resolving any object-category uncertainty can often help the next placement decision. However, random querying does not transfer to the longer tomato workflow: it achieves only 10.0% success despite requiring 8.9 queries on average and maintaining a query rate of 0.48. This domain gap indicates that query selection becomes more important when uncertainty can propagate across multiple dependent actions.

In contrast, **Ours** provides a favorable balance between task success and interaction cost. In waste sorting, it reduces the average number of queries from 15.9 to 8.3, but this reduction comes with a success-rate decrease from 97.5% for **All** to 91.5% for **Ours**. In tomato harvesting, **Ours** slightly outperforms **All**, achieving 84.5% success while reducing the average number of queries from 47.3 to 27.3. The query rates of 0.38 and 0.48 indicate that only a subset of uncertain decision points require human feedback. These results support a trade-off interpretation: **Ours** does not always maximize success rate, but it maintains high task performance while substantially reducing interaction.

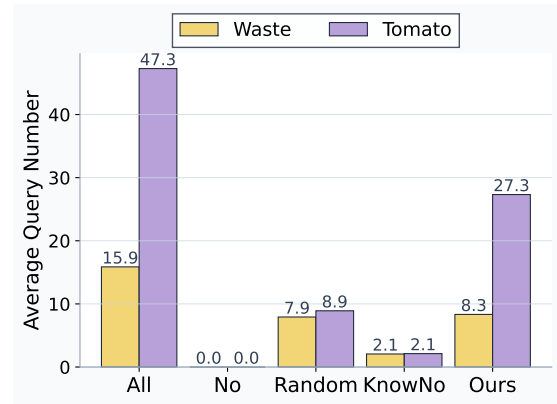
KnowNo exhibits a different trade-off. It achieves the lowest query rates among all feedback-based methods, requiring only

TABLE II: Threshold sensitivity for all evaluated values of τ . Success rates are percentages. Query counts are average questions per episode, query rate is the fraction of execution steps with at least one question, and average steps are computed over successful runs. Each domain-threshold cell aggregates 200 episodes.

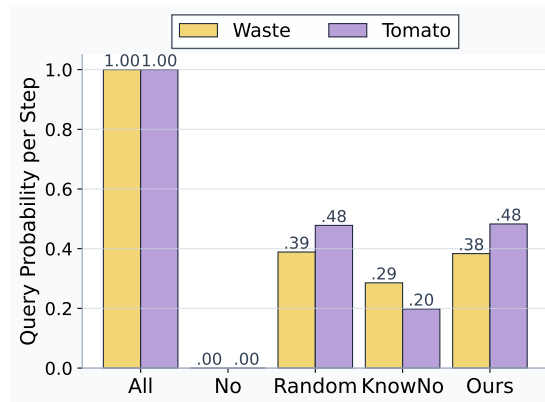
τ	Waste Sorting				Tomato Harvesting			
	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps
0.0	12.0	0.0	0.00	11.4	1.0	0.0	0.00	16.5
0.1	11.5	0.1	0.00	11.3	1.5	0.3	0.02	15.3
0.2	82.0	4.8	0.18	11.1	0.5	0.2	0.02	12.0
0.3	80.5	4.5	0.17	11.3	3.5	0.8	0.04	14.1
0.4	86.5	5.0	0.20	10.9	12.0	3.8	0.30	15.9
0.5	86.0	5.3	0.21	11.3	22.5	7.7	0.38	16.2
0.6	89.0	6.9	0.34	11.1	23.5	9.3	0.41	17.7
0.7	93.0	8.2	0.39	11.5	48.0	14.0	0.44	17.4
0.8	91.5	8.4	0.38	11.2	82.5	29.4	0.49	20.6
0.9	91.0	8.8	0.41	11.5	82.5	28.1	0.49	19.5
1.0	95.5	15.3	1.00	11.2	84.0	48.3	1.00	23.2



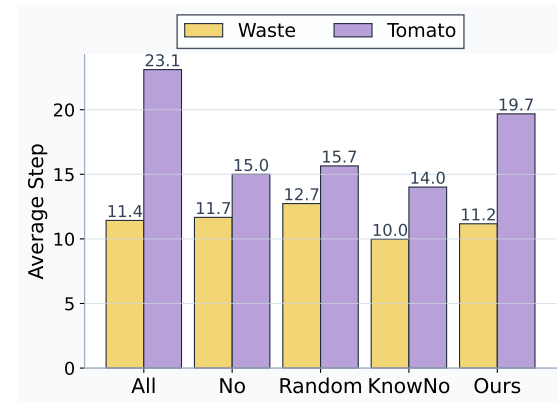
(a) Success rate



(b) Average queries



(c) Query rate



(d) Planning length

Fig. 7: Baseline comparison at $\tau = 0.8$. (a) Task success rate. (b) Average number of questions per episode. (c) Query rate, computed as the fraction of execution steps in which at least one question is asked. (d) Planning length. The proposed policy preserves high task success while asking substantially fewer questions than dense supervision.

2.1 queries on average in both domains. It also records the shortest successful execution sequences, with average planning lengths of 10.0 and 14.0 steps in waste sorting and tomato harvesting, respectively. Consequently, the overall amount of user interaction remains substantially lower than both **All** and **Ours**. Unlike state-level feedback, action-level feedback directly influences the planning process. By selecting the next action, the user provides information while simultaneously

guiding future execution. This guidance reduces both planning length and query frequency, leading to fewer subsequent queries and shorter execution trajectories.

However, this efficiency does not translate into reliable task completion in our domains. The success rates remain limited to 38.5% and 42.0%. Although action-level guidance can efficiently steer execution, it does not consistently resolve the underlying state uncertainty. Users must choose among

TABLE III: Baseline comparison at $\tau = 0.8$. Average queries and query rate are computed over all episodes, while average steps are computed over successful runs.

Condition	Waste Sorting				Tomato Harvesting			
	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps
All	97.5	15.9	1.00	11.4	81.5	47.3	1.00	23.1
No	13.5	0.0	0.00	11.7	1.5	0.0	0.00	15.0
Random	90.0	7.9	0.39	12.7	10.0	8.9	0.48	15.7
KnowNo	38.5	2.1	0.29	10.0	42.0	2.1	0.20	14.0
Ours	91.5	8.3	0.38	11.2	84.5	27.3	0.48	19.7

TABLE IV: Scalability results for **Ours** on six-object scenes. Steps are computed over successful runs, and the other average values are computed over all episodes.

Metric	Waste		Tomato	
	Orig.	Scale	Orig.	Scale
Success (%)	90.8	92.0	83.0	74.0
Steps	11.4	15.3	19.6	33.1
Queries	8.7	11.3	28.4	80.6
Query rate	0.41	0.33	0.49	0.54
Episode time (s)	2.1	22.9	10.1	159.9
Step time (s)	0.20	1.67	0.52	4.96
Expanded nodes	324.00	391.89	411.05	538.56
Avg. belief frontier size	36.13	68.97	10.99	74.32

candidate actions without direct access to the planner’s belief state, whereas state-level questions focus on observable facts that can be verified directly. We report the KnowNo implementation details in Appendix C to clarify the baseline setup and the prompt configuration used in this comparison.

3) *Scalability Test*: We next examine whether the proposed framework remains effective as task complexity increases. The original setting contains four objects per domain. The scaled setting contains six objects in both domains, increasing the symbolic belief space, the number of possible object assignments, and the planning complexity. We keep the policy fixed to **Ours** with $\tau = 0.8$ and increase the maximum planning horizon to accommodate the longer task sequences.

The scalability results show that the proposed policy remains effective as the number of task objects increases from four to six, but the impact differs by domain. In waste sorting, the success rate remains stable, increasing slightly from 90.8% to 92.0%. The number of successful-run steps increases from 11.4 to 15.3, and the average number of queries increases from 8.7 to 11.3. At the same time, the query rate decreases from 0.41 to 0.33, suggesting that the larger scene mainly lengthens the task rather than increasing the fraction of steps that require feedback.

The tomato harvesting domain presents a more challenging scalability test because uncertainty is distributed across multiple state variables, including object identity, stem location, ripeness, scan outcomes, and grasped objects. As the number of tomatoes increases, the success rate decreases from 83.0% to 74.0%, while the average plan length increases from 19.6 to 33.1 steps. The average number of queries also rises from 28.4 to 80.6. Despite this increase, the query rate changes only moderately from 0.49 to 0.54. This result suggests that the policy continues to request feedback selectively. The primary

scalability cost appears in planning and belief maintenance, as reflected by the increase in episode execution time from 10.1 to 159.9 seconds.

Overall, the scalability results suggest that larger symbolic scenes make the task substantially harder, increasing planning time and reducing success in the more complex tomato domain. Nevertheless, the proposed policy preserves selective feedback acquisition: even as the number of queries increases, the query rate grows only moderately. This indicates that the method requests feedback when uncertainty becomes important for planning. As a result, the robot can maintain stronger task performance than it would achieve without targeted human input.

4) *System Discussion*: The system evaluation highlights two key questions in human-robot interaction. The first is *when* the robot should ask for feedback. The second is *what* information it should request. The results suggest that both decisions are important for maintaining task performance while reducing user interaction.

The threshold experiment shows that more feedback is not always better. Low thresholds leave important uncertainty unresolved and lead to poor task performance. High thresholds increase the number of questions but do not provide proportional gains in success rate. The best performance is obtained when feedback is requested only for uncertainty that is likely to affect future planning decisions.

The baseline comparison shows that asking the right question is as important as deciding when to ask. **All** achieves high success rates but requires frequent interaction. **Random** also asks many questions, yet often fails because the selected questions are unrelated to the uncertainty that drives the task. **KnowNo** reduces both planning length and interaction by allowing users to directly guide action selection. However, action-level guidance does not reliably resolve the underlying state uncertainty, resulting in substantially lower task success. In contrast, **Ours** focuses on state uncertainty that directly affects planning and requests feedback only when that uncertainty becomes important. This allows the system to maintain high success rates while avoiding unnecessary questions.

The scalability experiment shows that larger problems primarily increase planning and belief maintenance costs. In the waste sorting domain, performance remains stable as the number of objects increases. In the tomato harvesting domain, the larger belief space leads to longer planning horizons, more queries, and higher computational cost. However, the query rate increases only moderately, indicating that the policy remains selective rather than asking for feedback at every

opportunity. The results demonstrate that the proposed query policy maintains efficient and selective feedback acquisition even as task complexity and belief-space size increase.

One limitation is that the frontier weights and observation likelihoods depend on domain-model probabilities. These parameters encode expected perception and execution reliability, and inaccurate calibration can affect both the confidence estimate and the resulting query behavior. In the present evaluation, we use fixed domain parameters to isolate the effect of the query policy; robustness to misspecified transition or observation probabilities remains an important direction for future work.

C. User Study

We conducted a Wizard-of-Oz (WoZ) user study to examine whether the proposed state-level query interface is usable with human participants, as a supplementary feasibility check alongside the system-level evaluation. Twelve participants each experienced every domain-condition pair once; we report task success, operation time, NASA Raw-TLX workload, and SAGAT accuracy.

1) *Participants*: Twelve participants (7 female and 5 male) took part in the study. Participants ranged in age from 21 to 26 years ($M = 23.0$, $SD = 1.48$). Each participant received \$20 upon completion of the study.

2) *Study Design and Conditions*: The study employed a within-subject design. Each participant completed ten trials in total, consisting of five waste sorting scenarios and five tomato harvesting scenarios. In each domain, participants experienced five interaction conditions: **All**, **No**, **Ours1**, **Ours2**, and **KnowNo**. **All** requested feedback whenever a query was available, representing continuous human supervision. **No** performed the task without requesting user feedback. **Ours1** used the proposed uncertainty-based state query policy with the same interaction logic as the system-level **Ours** policy. **Ours2** used the same state-level query interface with a different WoZ scenario profile to expose participants to a broader range of state-query situations. **KnowNo** served as an action-level query baseline, in which users were asked to select the robot's next action rather than verify a task-level state hypothesis. In the WoZ scripts, the action options in the **KnowNo** condition included the intended correct action, so this condition measured whether participants could identify the appropriate action from the interface rather than re-evaluating the autonomous **KnowNo** policy.

Each domain consisted of five scenarios and five interaction conditions. The assignment between scenarios and conditions was randomized for each participant such that every participant experienced all five conditions exactly once per domain. Detailed scenario descriptions are provided in Appendix D.

3) *Materials and Procedure*: Participants interacted with the robot through a tablet-based interface. The interface displayed the current task state, visual sensing information, task-relevant objects, and robot-generated questions with selectable answer options. The waste sorting scenarios required participants to identify waste categories, including general waste, paper, cans, and plastic. The tomato harvesting scenarios

required participants to identify tomato states and support harvesting decisions, such as whether a target tomato was present, ripe, unripe, or rotten.

Before the experiment, participants received an explanation of the two task domains and completed practice interactions with the tablet interface. During the main session, they completed ten trials without assistance. If a participant selected an incorrect answer, the response was recorded as incorrect for evaluation purposes, but the scenario continued using the intended task state so that later interactions were not affected by earlier mistakes. After each scenario, participants completed questionnaire items for workload and situation awareness.

4) *Measures and Analysis*: Task success measured whether participants provided all required responses correctly within a task. **No** is omitted from task success because it contains no scorable user responses. Operation time measured active user operation time after subtracting passive scenario video duration; **No** is also omitted from this measure because it involves no active interaction. Workload was measured using NASA Raw-TLX. We still report **No** for workload because no feedback is expected to impose the least interaction burden, but this condition does not provide user corrections to the robot. Situation awareness was measured using SAGAT-style questions, grouped into Lv1 perception, Lv2 comprehension, and Lv3 projection.

For condition-level comparisons, we used Cochran's Q test for task success and Friedman tests for operation time and workload. Given the small sample size and one trial per condition in each domain, we use these tests to qualify the descriptive results rather than to make broad claims about population-level effects. The statistical tests are exploratory and are reported without correction for multiple comparisons; pairwise post-hoc tests are not used to claim condition-level superiority.

5) *Task Success and Operation Time*: Table V and Fig. 8 summarize task success and operation time. In waste sorting, the proposed state-query conditions achieved high descriptive success rates (91.7% for **Ours1** and 100.0% for **Ours2**) with shorter operation time than **All**, while **KnowNo** showed lower task success. In tomato harvesting, **KnowNo** achieved the highest descriptive success rate, while the two proposed conditions required less operation time than **All**.

6) *Workload and SAGAT*: Fig. 9 and Table VI summarize NASA Raw-TLX workload. The workload scores were generally low across conditions. The **No** condition had the lowest workload in both domains, but this condition also provided no corrective support to the robot and is therefore not directly comparable to feedback-based conditions. Among feedback-based conditions, the proposed state-query conditions showed workload values comparable to or lower than **All** in several cases, without imposing a large subjective burden.

7) *SAGAT Results*: Table VII reports SAGAT accuracy by level. The results are mixed and do not show a consistent advantage for a single condition: some proposed-query conditions scored well on selected levels, but the pattern varied across domains and SAGAT levels. The **No** condition also obtained relatively high scores in some cases, which should

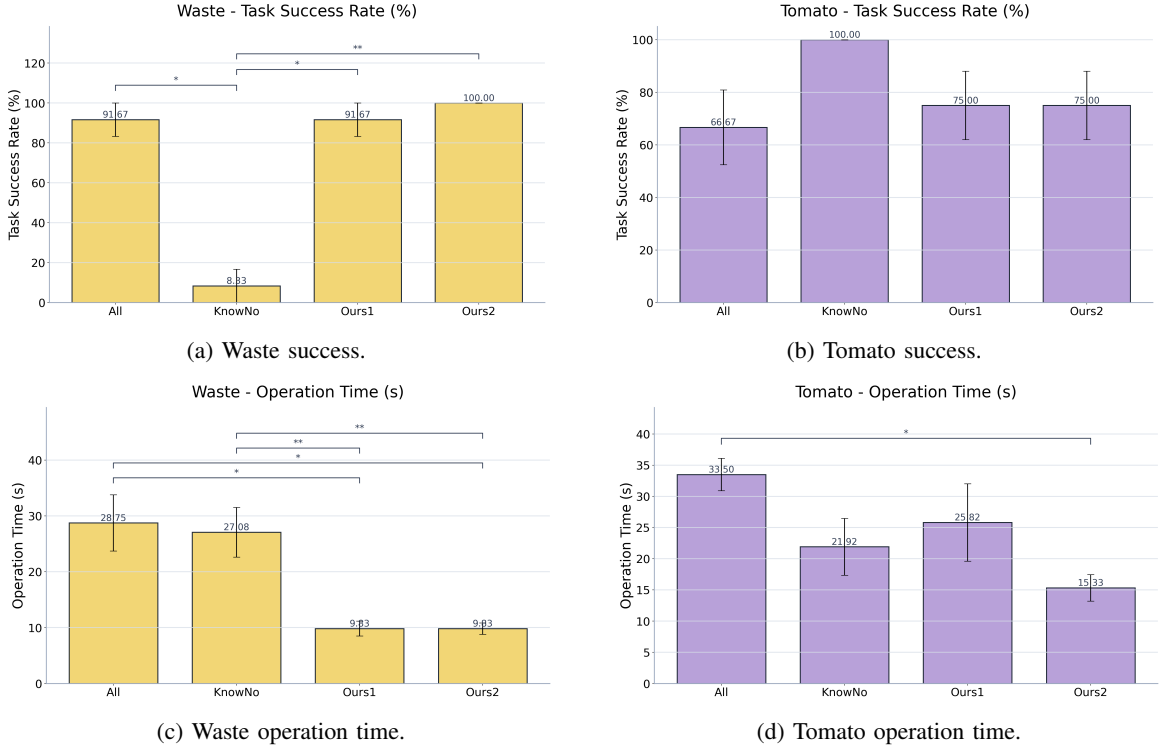


Fig. 8: Task success and operation time across query strategies. **No** is omitted from these two measures because they focus on scorable user responses and active user operation. Error bars indicate standard error.

TABLE V: Task success and operation time by domain and condition. Values are mean $\pm$ standard error.

Domain	Measure	All	Ours1	Ours2	KnowNo
Waste	Task success (%)	91.7 $\pm$ 8.3	91.7 $\pm$ 8.3	100.0 $\pm$ 0.0	8.3 $\pm$ 8.3
Waste	Operation time (s)	30.0 $\pm$ 6.7	11.2 $\pm$ 1.4	10.0 $\pm$ 1.3	29.2 $\pm$ 5.7
Tomato	Task success (%)	66.7 $\pm$ 14.2	75.0 $\pm$ 13.1	75.0 $\pm$ 13.1	100.0 $\pm$ 0.0
Tomato	Operation time (s)	35.2 $\pm$ 3.2	21.4 $\pm$ 3.5	17.6 $\pm$ 2.2	25.9 $\pm$ 5.4

TABLE VI: NASA Raw-TLX workload by domain and condition. Values are mean $\pm$ standard error.

Domain	All	No	Ours1	Ours2	KnowNo
Waste	2.06 $\pm$ 0.24	1.65 $\pm$ 0.16	2.26 $\pm$ 0.28	1.89 $\pm$ 0.21	2.54 $\pm$ 0.29
Tomato	2.21 $\pm$ 0.24	1.72 $\pm$ 0.19	1.89 $\pm$ 0.20	1.81 $\pm$ 0.15	1.81 $\pm$ 0.17

be interpreted cautiously because this condition involved observation without corrective feedback.

8) *Discussion*: This study served as a preliminary usability check for the proposed query interface alongside the main system-level evaluation in Section V-B. Participants were able to use state-level queries in both task domains, and the proposed conditions often reduced operation time or workload relative to continuous feedback, though outcomes varied by domain and measure.

The main user-study result is that state-level queries preserved task performance with less active interaction than continuous feedback. In waste sorting, the proposed state-query conditions achieved task success comparable to continuous feedback while substantially reducing operation time. This pattern supports the interface design: users answered targeted

questions about observable task facts, such as the category of a highlighted object, rather than continuously supervising the robot or inferring the planner's next action. By contrast, the **KnowNo** action-query condition required participants to choose the next action, which could require reasoning about hidden category uncertainty and the planner's need for additional detection. This distinction was especially important in waste sorting, where the visual scene could look actionable even when the planner still needed category information.

Tomato harvesting showed a different pattern. The action-level baseline **KnowNo** achieved the highest descriptive task success, suggesting that humans can select appropriate actions when the correct action is included in a constrained action set and the task follows a clear sequence of navigation, detection, picking, scanning, and placing or discarding. This does not mean that the autonomous KnowNo policy performed better in the system evaluation. The system-level baseline evaluated whether the policy could generate and use action-level queries during autonomous execution, whereas the WoZ condition evaluated whether participants could identify the intended action from controlled options. The result therefore highlights a boundary condition for action-level queries: they can be

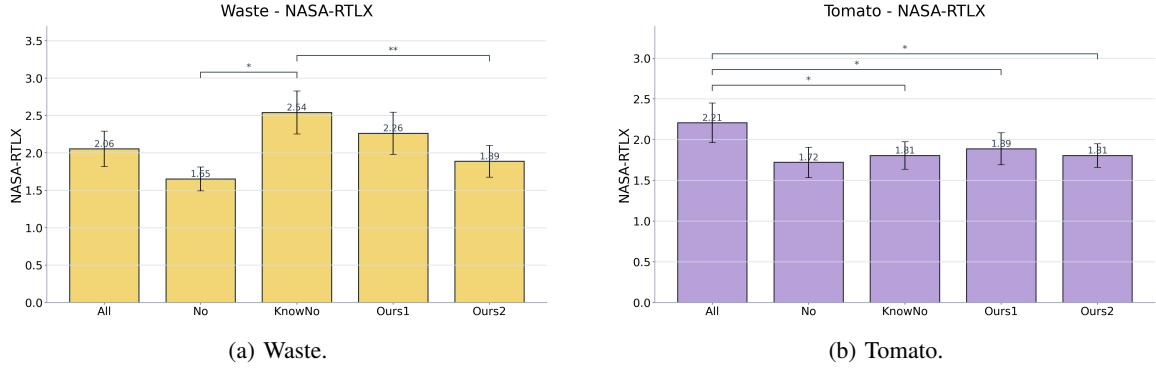


Fig. 9: NASA Raw-TLX workload. Lower values indicate lower workload. Error bars indicate standard error.

TABLE VII: SAGAT level accuracy by domain and condition. Values are mean $\pm$ standard error. Lv1, Lv2, and Lv3 correspond to perception, comprehension, and projection, respectively.

Domain	Condition	Lv1	Lv2	Lv3
Waste	All	0.92 $\pm$ 0.06	0.38 $\pm$ 0.13	0.38 $\pm$ 0.07
Waste	No	0.96 $\pm$ 0.04	0.25 $\pm$ 0.12	0.79 $\pm$ 0.10
Waste	Ours1	0.79 $\pm$ 0.10	0.63 $\pm$ 0.07	0.75 $\pm$ 0.08
Waste	Ours2	0.88 $\pm$ 0.07	0.08 $\pm$ 0.06	0.63 $\pm$ 0.09
Waste	KnowNo	0.75 $\pm$ 0.08	0.46 $\pm$ 0.13	0.58 $\pm$ 0.12
Tomato	All	0.88 $\pm$ 0.07	0.46 $\pm$ 0.10	0.71 $\pm$ 0.10
Tomato	No	0.75 $\pm$ 0.08	0.29 $\pm$ 0.10	0.83 $\pm$ 0.09
Tomato	Ours1	0.63 $\pm$ 0.07	0.42 $\pm$ 0.08	0.83 $\pm$ 0.07
Tomato	Ours2	0.67 $\pm$ 0.07	0.46 $\pm$ 0.13	0.96 $\pm$ 0.04
Tomato	KnowNo	0.63 $\pm$ 0.07	0.50 $\pm$ 0.09	0.63 $\pm$ 0.07

effective when the task structure and interface make the correct next action easy to infer, but they are more fragile when the user must reason about hidden state uncertainty.

The SAGAT results are exploratory but suggest that the distribution of state queries may shape different aspects of situation awareness. In waste sorting, **Ours1** showed relatively strong Lv2 comprehension, suggesting that targeted state verification can help users understand the current task situation. In tomato harvesting, **Ours2** showed strong Lv3 projection, suggesting that broader exposure to state-query situations may help users anticipate subsequent task behavior. These patterns should be interpreted cautiously because each participant experienced each domain-condition pair only once, but they indicate that query content may affect not only response efficiency but also the structure of user awareness.

Overall, the user study should not be read as a broad human-factors superiority claim. Workload scores were generally low across conditions, and the controlled WoZ scenarios were short. Instead, the study provides supplementary evidence for the system-level results: selective state queries can be implemented as a usable human interface, can reduce active interaction time relative to continuous feedback, and can avoid some of the inference burden imposed by action-level queries. A central design goal of the proposed interface is to shift the user's role from continuous monitoring to targeted verification: when the system detects relevant uncertainty, it asks the user to judge a specific state fact rather than infer the full planner state.

D. Real-Robot Implementation

We additionally implemented the proposed framework on the tomato harvesting robot platform to verify that the planning and query pipeline can be integrated with physical sensing and execution.

The robot platform consists of a Summit mobile base equipped with a UR5e manipulator, an RGB-D camera for object perception, and a LiDAR sensor for localization. The tomato domain follows the same symbolic task model used in the controlled evaluation. The robot maintains a belief over task-relevant predicates such as tomato location, ripeness, and whether an object has already been picked or processed. Visual detections are converted into symbolic observations, and the planner selects actions such as navigation, detection, picking, scanning, placing, and discarding.

The implementation was conducted in a tomato harvesting environment containing multiple tomato plants and collection basket, as shown in Fig. 10. The symbolic planner, belief manager, perception module, and query interface were integrated with the robot control system. Object detections obtained from the RGB-D camera were converted into symbolic predicates and provided to the planner, while navigation and manipulation commands were executed through the robot middleware.

During execution, the robot successfully performed navigation, tomato detection, harvesting, scanning, and placement behaviors while interacting with the user through the proposed query interface. User responses were incorporated into the planning process and influenced subsequent task decisions. The experiment demonstrates that the proposed framework can

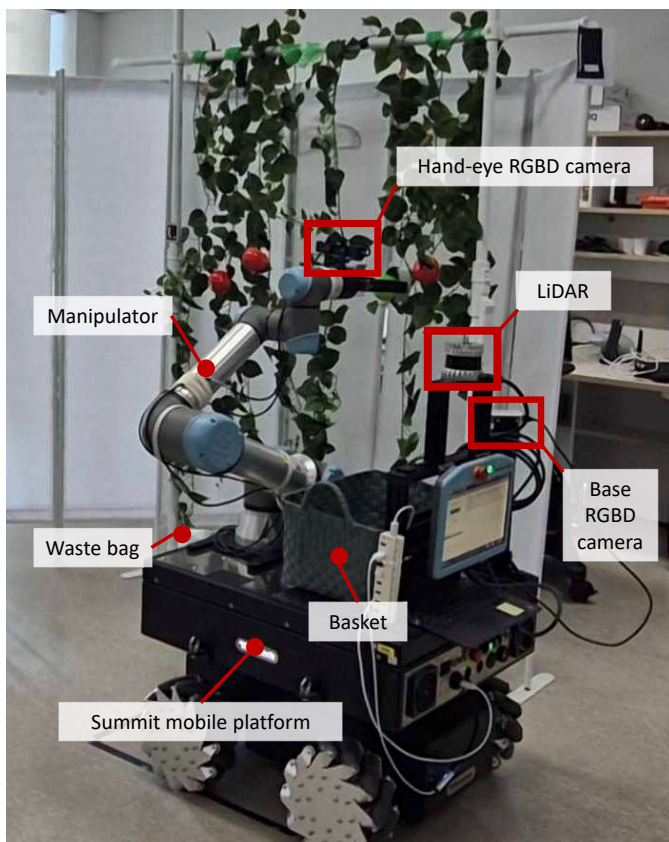


Fig. 10: Real-robot implementation in the tomato harvesting domain.

be deployed on a physical robot and integrated with real-world perception and execution pipelines.²

VI. CONCLUSION AND FUTURE WORK

In this paper, we proposed a robotic decision-making framework that integrates symbolic knowledge and uncertainty, with the goal of reducing user workload. The proposed method maintains a belief over possible worlds within a POMDP-based planning framework and quantitatively models uncertainty through a probability distribution over these worlds. Based on this, the robot can assess its own uncertainty and selectively query the human only when necessary. This approach enables stable task execution without continuous supervision and minimizes unnecessary interactions, thereby reducing user intervention.

In particular, we leverage entropy-based confidence to determine when to issue queries and select the most informative hypothesis to construct questions, improving the efficiency of interaction. Experimental results show that the proposed method reduces the number of human interactions while maintaining or improving task success rates, and effectively lowers user workload. User studies further support this trend, demonstrating that a system which actively manages uncertainty and requests assistance only when needed can operate effectively in real-world settings.

²Please see the supplementary video for the real-robot execution.

These results point to several extensions. First, richer feedback channels can build on the current binary hypothesis interface by allowing users to provide corrected symbolic facts or natural-language clarifications. Second, scalable frontier construction can extend the approach to larger domains through pruning, abstraction, or learned proposal distributions. Third, user-aware query selection can combine robot belief uncertainty with estimates of what users can judge from the interface. Finally, the framework can be extended to multi-robot settings, where multiple agents share uncertainty and coordinate when and whom to query. These extensions preserve the central idea of the framework: human feedback is most useful when it is requested at belief-relevant moments and expressed in a form that supports both planning and human understanding.

ACKNOWLEDGEMENT

This research was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(No. RS-2024-00461583) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00411007).

REFERENCES

- [1] K. N. Kaipa, A. S. Kankanhalli-Nagendra, N. B. Kumbala, S. Shriyam, S. S. Thevendria-Karthic, J. A. Marvel, and S. K. Gupta, "Addressing perception uncertainty induced failure modes in robotic bin-picking," *Robotics and Computer-Integrated Manufacturing*, vol. 42, pp. 17–38, 2016.
- [2] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [3] H. Kurniawati and V. Yadav, "An online pomdp solver for uncertainty planning in dynamic environment," in *Proc. of International Symposium of Robotics Research (ISRR)*. Springer, 2016, pp. 611–629.
- [4] A. D. Dragan, K. C. Lee, and S. S. Srinivasa, "Legibility and predictability of robot motion," in *Proc. of International Conference on Human-Robot Interaction (HRI)*. IEEE, 2013, pp. 301–308.
- [5] M. A. Goodrich and A. C. Schultz, "Human-robot interaction: a survey," *Foundations and trends® in human-computer interaction*, vol. 1, no. 3, pp. 203–275, 2008.
- [6] J. Y. Chen and M. J. Barnes, "Human-agent teaming for multi-robot control: A review of human factors issues," *Transactions on Human-Machine Systems*, vol. 44, no. 1, pp. 13–29, 2014.
- [7] D. Sadigh, A. D. Dragan, S. S. Sastry, and S. A. Seshia, "Active preference-based learning of reward functions," in *Proc. of Robotics: Science and Systems (RSS)*, 2017. [Online]. Available: <https://escholarship.org/uc/item/88k894w7>
- [8] J. Leusmann, C. Wang, M. Gienger, A. Schmidt, and S. Mayer, "Understanding the uncertainty loop of human-robot interaction," *arXiv preprint arXiv:2303.07889*, 2023.
- [9] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley et al., "Robots that ask for help: Uncertainty alignment for large language model planners," *arXiv preprint arXiv:2307.01928*, 2023.
- [10] K. Liang, Z. Zhang, and J. F. Fisac, "Introspective planning: Aligning robots' uncertainty with inherent task ambiguity," *Advances in Neural Information Processing Systems*, vol. 37, pp. 71 998–72 031, 2024.
- [11] J. F. Mullen and D. Manocha, "Lbap: Improved uncertainty alignment of llm planners using bayesian inference," in *2025 International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 18 716–18 723.
- [12] R. Davis, H. Shrobe, and P. Szolovits, "What is a knowledge representation?" *AI magazine*, vol. 14, no. 1, pp. 17–17, 1993.
- [13] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.

- [14] M. Tenorth and M. Beetz, "Knowledge processing for autonomous robot control," in *Proc. of Spring Symposium on Designing Intelligent Robots (AAAI)*, 2012.
- [15] Z. Liu, M. Jiang, and H. Lin, "A graph-based spatial temporal logic for knowledge representation and automated reasoning in cognitive robots," *arXiv preprint arXiv:2001.07205*, 2020.
- [16] L. Kunze, K. K. Doreswamy, and N. Hawes, "Using qualitative spatial relations for indirect object search," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2014, pp. 163–168.
- [17] S. Hasegawa, A. Taniguchi, Y. Hagiwara, L. El Hafi, and T. Taniguchi, "Inferring place-object relationships by integrating probabilistic logic and multimodal spatial concepts," in *Proc. of International Symposium on System Integration (SII)*, 2023, pp. 1–8.
- [18] T. Mota, M. Sridharan, and A. Leonardis, "Integrated commonsense reasoning and deep learning for transparent decision making in robotics," *SN Computer Science*, vol. 2, no. 4, p. 242, 2021.
- [19] S. Zhang, P. Khandelwal, and P. Stone, "iCORPP: Interleaved commonsense reasoning and probabilistic planning on robots," *Robotics and Autonomous Systems*, vol. 174, p. 104613, 2024.
- [20] A. Daruna, W. Liu, Z. Kira, and S. Chetnova, "RoboCSE: Robot common sense embedding," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2019, pp. 9777–9783.
- [21] A. Mitrevsk, P. G. Plöger, and G. Lakemeyer, "Ontology-assisted generalisation of robot action execution knowledge," in *2021 International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 6763–6770.
- [22] A. Olivares-Alarcos, D. Beßler, A. Khamis, P. Goncalves, M. K. Habib, J. Bermejo-Alonso, M. Barreto, M. Diab, J. Rosell, J. Quintas et al., "A review and comparison of ontology-based approaches to robot autonomy," *The Knowledge Engineering Review*, vol. 34, p. e29, 2019.
- [23] S. Manzoor, Y. G. Rocha, S.-H. Joo, S.-H. Bae, E.-J. Kim, K.-J. Joo, and T.-Y. Kuc, "Ontology-based knowledge representation in robotic systems: A survey oriented toward applications," *Applied Sciences*, vol. 11, no. 10, p. 4324, 2021.
- [24] C. Aeronautiques, A. Howe, C. Knoblock, I. D. McDermott, A. Ram, M. Veloso, D. Weld, D. W. Sri, A. Barrett, D. Christianson et al., "PDDL—the planning domain definition language," *Technical Report, Tech. Rep.*, 1998.
- [25] M. Helmert, "Concise finite-domain representations for pddl planning tasks," *Artificial Intelligence*, vol. 173, no. 5-6, pp. 503–535, 2009.
- [26] J. Hoffmann, "Ff: The fast-forward planning system," *AI magazine*, vol. 22, no. 3, pp. 57–77, 2001.
- [27] M. Helmert, "The fast downward planning system," *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [28] A. B. Corrêa and G. De Giacomo, "Lifted planning: Recent advances in planning using first-order representations," in *Proc. of International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 8010–8019.
- [29] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman et al., "Do as i can, not as i say: Grounding language in robotic affordances," 2022.
- [30] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar et al., "Inner monologue: Embodied reasoning through planning with language models," 2022.
- [31] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Program generation for situated robot task planning using large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 999–1012, 2023.
- [32] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.
- [33] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *Proc. of International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2086–2092.
- [34] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa et al., "Concept-graphs: Open-vocabulary 3d scene graphs for perception and planning," in *Proc. of International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5021–5028.
- [35] A. Leanza, A. Moroncelli, G. Vizzari, F. Braghin, L. Roveda, and B. Spahiu, "Conceptbot: Enhancing robot's autonomy through task decomposition with large language models and knowledge graph," *arXiv preprint arXiv:2509.00570*, 2025.
- [36] S. Singh, K. Swaminathan, N. Dash, R. Singh, S. Banerjee, M. Sridharan, and M. Krishna, "Adaptbot: Combining llm with knowledge graphs and human input for generic-to-specific task decomposition and knowledge refinement," in *2025 International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4345–4351.
- [37] J. Zhou, J. Miao, X. Wang, and J. Yu, "Enhancing llm planning for robotics manipulation through hierarchical procedural knowledge graphs," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [38] W. Xu, M. Wang, W. Zhou, and H. Li, "P-RAG: Progressive retrieval augmented generation for planning on embodied everyday task," in *Proc. of ACM International Conference on Multimedia (MM)*, 2024, pp. 6969–6978.
- [39] Z. Wang, S. X. Teo, J. J. Chew, and W. Shi, "InstructRAG: Leveraging retrieval-augmented generation on instruction graphs for llm-based task planning," in *Proc. of International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2025.
- [40] N. Kumar, W. Shen, F. Ramos, D. Fox, T. Lozano-Pérez, L. P. Kaelbling, and C. R. Garrett, "Open-world task and motion planning via vision-language model generated constraints," *Robotics and Automation Letters*, 2026.
- [41] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3d value maps for robotic manipulation with language models," in *Proc. of Conference on Robot Learning (CoRL)*, 2023, pp. 540–562.
- [42] Z. Dai, A. Asgharivaskasi, T. Duong, S. Lin, M.-E. Tzes, G. Pappas, and N. Atanasov, "Optimal scene graph planning with large language model guidance," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14062–14069.
- [43] Y. Liu, L. Palmieri, S. Koch, I. Georgievski, and M. Aiello, "DELTA: Decomposed efficient long-term robot task planning using large language models," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2025, pp. 10995–11001.
- [44] D. Paulius, A. Agostini, B. Quartey, and G. Konidaris, "Bootstrapping object-level planning with large language models," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2025, pp. 16233–16239.
- [45] B. Quartey, E. Rosen, S. Tellex, and G. Konidaris, "Verifiably following complex robot instructions with foundation models," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2025, pp. 1–8.
- [46] A. Somani, N. Ye, D. Hsu, and W. S. Lee, "Despot: Online pomdp planning with regularization," *Advances in neural information processing systems*, vol. 26, 2013.
- [47] H. Bai, S. Cai, N. Ye, D. Hsu, and W. S. Lee, "Intention-aware online pomdp planning for autonomous driving in a crowd," in *2015 international conference on robotics and automation (icra)*. IEEE, 2015, pp. 454–460.
- [48] M. Cramer, K. Kellens, and E. Demeester, "Probabilistic decision model for adaptive task planning in human-robot collaborative assembly based on designer and operator intents," *Robotics and Automation Letters*, vol. 6, no. 4, pp. 7325–7332, 2021.
- [49] R. Z. B. Bravo, A. Leiras, and F. L. Cyrino Oliveira, "The use of UAVs in humanitarian relief: An application of pomdp-based methodology for finding victims," *Production and Operations Management*, vol. 28, no. 2, pp. 421–440, 2019.
- [50] M. Lauri and R. Ritala, "Planning for robotic exploration based on forward simulation," *Robotics and Autonomous Systems*, vol. 83, pp. 15–31, 2016.
- [51] S.-K. Kim, O. Salzman, G. Agarwal et al., "PLGRIM: Hierarchical value learning for large-scale exploration in unknown environments," in *Proc. of International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31, 2021, pp. 652–662.
- [52] F. Vanegas, D. Campbell, M. Eich, and F. Gonzalez, "UAV based target finding and tracking in GPS-denied and cluttered environments," in *Proc. of the International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 2307–2313.
- [53] Y. Xiao, S. Katt, A. ten Pas, S. Chen, and C. Amato, "Online planning for target object search in clutter under partial observability," in *Proc. of the International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8241–8247.
- [54] J. Pajarinen, J. Lundell, and V. Kyrki, "POMDP planning under object composition uncertainty: Application to robotic manipulation," *Transactions on Robotics*, vol. 39, no. 1, pp. 41–56, 2023.
- [55] M. Li, Z. Zhang, and B. Song, "Uncertainty-aware neural-symbolic task planning with balanced exploration and execution for domestic service robot," *Expert Systems with Applications*, p. 132962, 2026.

- [56] S. Sanner and K. Kersting, "Symbolic dynamic programming for first-order pomdps," in *Proc. of Conference on Artificial Intelligence (AAAI)*, vol. 24, no. 1, 2010, pp. 1140–1146.
- [57] S. Zhang and P. Stone, "Corpp: Commonsense reasoning and probabilistic planning, as applied to dialog with a mobile robot," in *Proc. of Conference on Artificial Intelligence (AAAI)*, 2015, pp. 469–475.
- [58] S. A. Serrano, E. Santiago, J. Martinez-Carranza, E. F. Morales, and L. E. Sucar, "Knowledge-based hierarchical pomdps for task planning," *Journal of Intelligent & Robotic Systems*, vol. 101, no. 4, p. 82, 2021.
- [59] W. Tang, X. He, Y. Huang, Y. Xiao, C. Lu, and P. Cai, "Tru-pomdp: Task planning under uncertainty via tree of hypotheses and open-ended pomdps," *arXiv preprint arXiv:2506.02860*, 2025.
- [60] S. Singi, Z. He, A. Pan, S. Patel, G. A. Sigurdsson, R. Piramuthu, S. Song, and M. Ciocarlie, "Decision making for human-in-the-loop robotic agents via uncertainty-aware reinforcement learning," in *Proc. of international conference on robotics and automation (ICRA)*. IEEE, 2024, pp. 7939–7945.
- [61] I. Igbinedion and S. Karaman, "Learning when to ask for help: Efficient interactive navigation via implicit uncertainty estimation," in *Proc. of International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 9593–9599.
- [62] M. Zhao, R. Simmons, H. Admoni, A. Ramdas, and A. Bajcsy, "Conformalized interactive imitation learning: Handling expert shift and intermittent feedback," in *Proc. of International Conference on Learning Representations (ICLR)*, 2025.
- [63] D. H. Mangalindan, E. Rovira, and V. Srivastava, "When to seek help: Trust-aware assistance-seeking in human-supervised autonomy," *Transactions on Human-Robot Interaction*, vol. 15, no. 2, pp. 1–23, 2025.
- [64] Y. Shen, D. Bis, C. Lu, and I. Lourentzou, "Elba: Learning by asking for embodied visual navigation and task completion," in *Proc. of the International Conference on Computer Vision (CVPR)*, 2025, pp. 5177–5186.
- [65] A. Testoni and R. Fernández, "Asking the right question at the right time: Human and model uncertainty guidance to ask clarification questions," in *Proc. of Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024, pp. 258–275.
- [66] R. Brachman and H. Levesque, *Knowledge representation and reasoning*. Elsevier, 2004.
- [67] S. Russell, P. Norvig, and A. Intelligence, "A modern approach," *Artificial Intelligence*. Prentice-Hall, Englewood Cliffs, vol. 25, no. 27, pp. 79–80, 1995.
- [68] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. Shamma, M. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proc. of the International Conference on Computer Vision (CVPR)*, 2015, pp. 3668–3678.
- [69] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proc. of the International Conference on Computer Vision (CVPR)*, 2019, pp. 5664–5673.
- [70] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, "NL2HLTL2PLAN: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task specifications," *Robotics and Automation Letters*, 2025.
- [71] B. Rabei, M. K. AR, Z. Dai, S. L. Pilla, Q. Dong, and N. Atanasov, "LTLCodeGen: Code generation of syntactically correct temporal logic for robot task planning," in *Proc. of International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 19240–19247.
- [72] Y. Wu, Z. Xiong, Y. Hu, S. S. Iyengar, N. Jiang, A. Bera, L. Tan, and S. Jagannathan, "SELP: Generating safe and efficient task plans for robot agents with large language models," in *Proc. of International Conference on Robotics and Automation (ICRA)*, 2025, pp. 2599–2605.
- [73] J. Li, M. Tian, and B. Zhong, "Automatic generation of safety-compliant linear temporal logic via large language model: A self-supervised framework," 2025.
- [74] S. Hu, T. Lin, K. Ye, Y. Yang, and T. Zhang, "LLM-grounded dynamic task planning with hierarchical temporal logic for human-aware multi-robot collaboration," 2026.
- [75] J. Pineau, G. Gordon, S. Thrun et al., "Point-based value iteration: An anytime algorithm for pomdps," in *Ijcai*, vol. 3, 2003, pp. 1025–1032.
- [76] S. Ross, J. Pineau, S. Paquet, and B. Chaib-Draa, "Online planning algorithms for pomdps," *Journal of Artificial Intelligence Research*, vol. 32, pp. 663–704, 2008.
- [77] D. Silver and J. Veness, "Monte-carlo planning in large pomdps," *Advances in neural information processing systems*, vol. 23, 2010.
- [78] P. Doshi and P. J. Gmytrasiewicz, "A particle filtering based approach to approximating interactive pomdps," in *Proc. of Conference on Artificial Intelligence (AAAI)*, 2005, pp. 969–974.
- [79] M. H. Lim, T. J. Becker, M. J. Kochenderfer, C. J. Tomlin, and Z. N. Sunberg, "Optimality guarantees for particle belief approximation of pomdps," *Journal of Artificial Intelligence Research*, vol. 77, pp. 1591–1636, 2023.
- [80] Z. N. Sunberg and M. J. Kochenderfer, "Online algorithms for POMDPs with continuous state, action, and observation spaces," in *Proc. of International Conference on Automated Planning and Scheduling (ICAPS)*, 2018, pp. 259–263.
- [81] S. Brechtel, T. Gindele, and R. Dillmann, "Probabilistic decision-making under uncertainty for autonomous driving using continuous POMDPs," in *Proc. of the International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2014, pp. 392–399.
- [82] H. Bai, D. Hsu, M. J. Kochenderfer, and W. S. Lee, "Unmanned aircraft collision avoidance using continuous-state POMDPs," in *Proc. of Robotics: Science and Systems (RSS)*, 2011, pp. 1–8.
- [83] J. Pajarinen and V. Kyrki, "Robotic manipulation of multiple objects as a POMDP," *Artificial Intelligence*, vol. 247, pp. 213–228, 2017.
- [84] E. A. Hansen and Z. Feng, "Dynamic programming for pomdps using a factored state representation," in *AIPS*, 2000, pp. 130–139.
- [85] Z. Feng and E. Hansen, "Approximate planning for factored pomdps," in *Proceedings of the 6th European Conference on Planning*, 2001.
- [86] T. Veiga, M. Spaan, and P. Lima, "Point-based pomdp solving with factored value function approximation," in *Proc. of Conference on Artificial Intelligence (AAAI)*, vol. 28, no. 1, 2014.
- [87] S. R. Eddy, "Hidden markov models," *Current opinion in structural biology*, vol. 6, no. 3, pp. 361–365, 1996.
- [88] K. Murphy and Y. Weiss, "The factored frontier algorithm for approximate inference in dbns," *arXiv preprint arXiv:1301.2296*, 2013.
- [89] M. Ghallab, D. Nau, and P. Traverso, *Automated planning and acting*. Cambridge University Press, 2016.
- [90] V. Lifschitz, *Answer set programming*. Springer Cham, 2019, vol. 3.
- [91] M. Richardson and P. Domingos, "Markov logic networks," *Machine learning*, vol. 62, no. 1, pp. 107–136, 2006.
- [92] M. Qu and J. Tang, "Probabilistic logic neural networks for reasoning," *Advances in neural information processing systems*, vol. 32, 2019.
- [93] H. Kurniawati, D. Hsu, W. S. Lee et al., "Sarsop: Efficient point-based pomdp planning by approximating optimally reachable belief spaces," in *Robotics: Science and systems*, vol. 2008. Zurich, Switzerland, 2008.
- [94] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *Proc. of International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8077–8083.
- [95] A. Mandlekar, D. Xu, R. Martín-Martín, Y. Zhu, L. Fei-Fei, and S. Savarese, "Human-in-the-loop imitation learning using remote tele-operation," *arXiv preprint arXiv:2012.06733*, 2020.
- [96] P. Scharre, "Introduction to autonomy in weapon systems," 2015.
- [97] S. Nahavandi, "Trusted autonomy between humans and robots: Toward human-on-the-loop in robotics and autonomous systems," *Systems, Man, and Cybernetics Magazine*, vol. 3, no. 1, pp. 10–17, 2017.
- [98] M. R. Endsley, "Automation and situation awareness," in *Automation and human performance*. CRC Press, 2018, pp. 163–181.
- [99] D. Davies and R. Parasuraman, *The psychology of violence*. Academic Press, New York, 1982, vol. 1.
- [100] D. R. Davies, G. Matthews, R. B. Stammers, and S. J. Westerman, *Human performance: Cognition, stress and individual differences*. Psychology Press, 2013.
- [101] A. Mack, "Inattention blindness: Looking without seeing," *Current directions in psychological science*, vol. 12, no. 5, pp. 180–184, 2003.
- [102] C. Y. Wong and G. Seet, "Workload, awareness and automation in multiple-robot supervision," *International Journal of Advanced Robotic Systems*, vol. 14, no. 3, p. 1729881417710463, 2017.
- [103] S. M. Galster, B. A. Knott, and R. D. Brown, "Managing multiple uavs: Are we asking the right questions?" in *Proc. of Human Factors and Ergonomics Society Annual Meeting (HFES)*, vol. 50, no. 3. SAGE Publications Sage CA: Los Angeles, CA, 2006, pp. 545–549.
- [104] J. Baraglia, M. Cakmak, Y. Nagai, R. P. Rao, and M. Asada, "Efficient human-robot collaboration: when should a robot take initiative?" *The International Journal of Robotics Research*, vol. 36, no. 5-7, pp. 563–579, 2017.
- [105] A. Murata, "Ergonomics and cognitive engineering for robot-human cooperation," in *Proc. of International Workshop on Robot and Human Interactive Communication (RO-MAN)*, 2000, pp. 206–211.

- [106] G. Borragán, H. Slama, M. Bartolomei, and P. Peigneux, “Cognitive fatigue: A time-based resource-sharing account,” *Cortex*, vol. 89, pp. 71–84, 2017.
- [107] A. Rajavenkatanarayanan, V. Kanal, K. Tsiakas, J. Brady, D. Calderon, G. Wylie, and F. Makedon, “Towards a robot-based multimodal framework to assess the impact of fatigue on user behavior and performance: A pilot study,” in *Proc. of International Conference on Pervasive Technologies Related to Assistive Environments (PETRA)*, 2019, pp. 493–498.
- [108] S. Chitta, I. Sucan, and S. Cousins, “Moveit![ros topics],” *robotics & automation magazine*, vol. 19, no. 1, pp. 18–19, 2012.
- [109] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” *arXiv preprint arXiv:2502.12524*, 2025.
- [110] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat et al., “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.

APPENDIX CONTENTS

This appendix provides supplementary materials for the planning algorithms, experimental settings, additional results, and user-study materials used in the main text.

- 1) Appendix A. Planning algorithms
 - Appendix A-A. POMCP search with applicable actions
 - Appendix A-B. POMCP simulation with applicable actions
 - Appendix A-C. Action selection and rollout
- 2) Appendix B. Detailed experimental settings
 - Appendix B-A. Planner and evaluation parameters
 - Appendix B-B. Domain model probabilities
 - Appendix B-C. Randomization and episode seeds
 - Appendix B-D. Scalability experiment settings
- 3) Appendix C. KnowNo baseline details
 - Appendix C-A. KnowNo baseline implementation
- 4) Appendix D. User-study scenarios and interfaces
 - Appendix D-A. Scenario structure and questionnaire timing
 - Appendix D-B. Condition codes and participant-level assignment
 - Appendix D-C. Questionnaires
 - Appendix D-D. Scenario-level answer summary

APPENDIX A PLANNING ALGORITHMS

This appendix provides the detailed pseudocode for the POMCP-based search routines described in Sec. IV-A2.

A. POMCP Search with Applicable Actions

Alg. 2 summarizes the root search procedure. The search samples states from the current belief when particles are available, falls back to the committed knowledge state when the belief is empty, and repeatedly calls the simulation routine until the computation budget is exhausted. The final action is selected from actions applicable to the current knowledge state.

Algorithm 2: POMCP Search with Applicable Actions

```

1 Input: belief  $b$ 
2 Output: selected action  $a \in \mathcal{A}$ 
3 procedure SEARCH( $b$ )
4    $h \leftarrow$  root history
5    $s^K \leftarrow \text{knowledge}(b)$ 
6   repeat
7     if  $B(h) \neq \emptyset$  then
8       | sample  $s \sim B(h)$ 
9     else
10      |  $s \leftarrow s^K$ 
11    SIMULATE( $s, h, 0$ )
12  until TIMEOUT;
13   $a \leftarrow \text{SEARCH-BEST}(h, s^K, \text{false})$ 
14  return  $a$ 

```

B. POMCP Simulation with Applicable Actions

Alg. 3 describes how simulations are constrained by symbolic action applicability. When a new history node is expanded, only actions whose preconditions hold in the sampled symbolic state are inserted into the search tree. The generative model then samples the next state, observation, and reward while preserving the standard POMCP value-update structure.

Algorithm 3: POMCP Simulation with Applicable Actions

```

1 Input: state  $s$ , history  $h$ , depth  $depth$ 
2 Output: simulated return  $R$ 
3 procedure SIMULATE ( $s, h, depth$ )
4   if  $\gamma^{depth} < \epsilon$  then
5     return 0
6   if there is no  $a[s] \in \mathcal{A}$  then
7      $N(h) \leftarrow N(h) + 1$ 
8     return 0
9   if  $h \notin T$  then
10    foreach  $a[s] \in \mathcal{A}$  do
11       $T, N(ha), V(ha) \leftarrow T \cup \{ha\}, 0, 0$ 
12     $N(h) \leftarrow N(h) + 1$ 
13    return ROLLOUT ( $s, depth$ )
14   $a \leftarrow \text{SEARCH-BEST}(h, s, \text{true})$ 
15  sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
16   $R \leftarrow r + \gamma \cdot \text{SIMULATE}(s', hao, depth + 1)$ 
17   $B(h) \leftarrow B(h) \cup \{s\}$ 
18   $N(h) \leftarrow N(h) + 1$ 
19   $N(ha) \leftarrow N(ha) + 1$ 
20   $V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}$ 
21  return  $R$ 

```

Algorithm 4: Action Selection and Rollout

```

1 Input: history  $h$ , state  $s$ 
2 procedure SEARCH-BEST ( $h, s, use\_ucb$ )
3    $\mathcal{C} \leftarrow \{a \mid a[s] \in \mathcal{A}, ha \in T\}$ 
4   if  $\mathcal{C} = \emptyset$  then
5     return  $\emptyset$ 
6   if  $use\_ucb$  then
7     return  $\arg \max_{a \in \mathcal{C}} V(ha) + c \cdot \sqrt{\frac{\log N(h)}{1 + N(ha)}}$ 
8   else
9     return  $\arg \max_{a \in \mathcal{C}} V(ha)$ 
10 procedure ROLLOUT ( $s, depth$ )
11   if  $(\gamma^{depth} < \epsilon \vee \gamma = 0) \wedge depth > 0$  then
12     return 0
13   if there is no  $a[s] \in \mathcal{A}$  then
14     return 0
15    $a \sim \text{Uniform}(\{a \mid a[s] \in \mathcal{A}\})$ 
16   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
17   return  $r + \gamma \cdot \text{ROLLOUT}(s', depth + 1)$ 

```

C. Action Selection and Rollout

Alg. 4 specifies the shared action-selection routine used during tree search and rollout. Tree actions are selected using either the UCB criterion or the current value estimate, whereas rollout actions are sampled uniformly from applicable actions. This keeps both planning and rollout consistent with the symbolic transition model.

APPENDIX B DETAILED EXPERIMENTAL SETTINGS

This appendix summarizes implementation details that support the experimental setup in Sec. V-A. The standard system-evaluation experiments use five scenario files per domain, `scene_01.yaml`–`scene_05.yaml`. For each threshold or policy condition, we evaluate all five scenarios with 40 randomized episodes per scenario, yielding 200 episodes per domain-condition cell. The maximum standard episode length is 50 steps, and all POMCP searches use 100 simulations per planning step, maximum simulation depth 20, discount factor $\gamma = 0.95$, and UCB exploration constant $c = 1.0$.

A. Planner and Evaluation Parameters

Table VIII lists the planner and evaluation parameters used for the standard system experiments. These values are shared across threshold and policy comparisons unless otherwise stated.

TABLE VIII: Planner and evaluation parameters used in the system experiments.

Parameter	Value
POMCP simulations per planning step	100
Maximum simulation depth	20
Discount factor γ	0.95
UCB exploration constant c	1.0
Maximum episode steps	50
Scenarios per domain	5
Episodes per scenario-condition	40
Episodes per domain-condition	200

B. Domain Model Probabilities

Table IX summarizes the transition and observation probabilities used by the domain models. Transition probabilities define uncertainty in action outcomes, while observation probabilities define the likelihood terms used in belief updates.

TABLE IX: Domain model probabilities used in the reported experiments.

Domain	Model component	Probability
Waste sorting	Detect observed transition	0.90
Waste sorting	Detect category transition	0.50
Waste sorting	Pick transition	0.90
Waste sorting	Place transition	0.90
Waste sorting	Detect observed observation	0.90
Waste sorting	Detect category observation	0.80
Waste sorting	Pick/place observation	0.95
Tomato harvesting	Navigate transition	0.90
Tomato harvesting	Detect transition	0.95
Tomato harvesting	Pick transition	0.95
Tomato harvesting	Scan transition	0.90
Tomato harvesting	Place/discard transition	0.99
Tomato harvesting	Detect observed observation	0.85
Tomato harvesting	Detect classification observation	0.95
Tomato harvesting	Scan observation	0.85

The waste-sorting detect category transition probability of 0.50 defines the controlled category-ambiguity profile used in the experiments. It creates competing category hypotheses after detection, allowing the planner and query policy to be evaluated in a setting where category uncertainty matters for bin placement. The observation model is specified separately from this transition uncertainty; for example, the waste detect category observation probability is 0.80 and is used when computing the likelihood term $O(o \mid s_k^*)$ during belief update.

C. Randomization and Episode Seeds

For each episode, the runner generates and records a pseudorandom seed. The seed is passed to the execution script and controls Python `random`, NumPy sampling, randomized initial arrangements, and the random-query policy when applicable. This allows individual raw episodes to be traced back to the scenario, threshold or policy condition, and sampled execution trajectory.

D. Scalability Experiment Settings

The scalability experiment uses the same planner parameters but evaluates the proposed policy on larger six-object scenarios, `scene_06.yaml`–`scene_10.yaml`. Each scaled scene is run for 40 randomized episodes, again yielding 200 episodes per domain. Because these scenarios have longer optimal plans, the maximum episode length is increased to 90 steps and the maximum number of belief particles is set to 800. The scalability table in Sec. V-B3 reports means over these scaled episodes and compares them with the corresponding standard **Ours** results at $\tau = 0.8$.

APPENDIX C KNOWNO BASELINE DETAILS

This appendix describes the KnowNo-style action-query baseline used in the policy comparison. The baseline uses the same domain scenarios, task dynamics, execution limits, and scene randomization protocol as the other policy baselines. The main experiments report the GPT-4o variant with the structured prompt version shown below, conformal threshold $\hat{q} = 0.92$, and scoring temperature 3.0. This setup follows the action-query framing of KnowNo while keeping the environment, transition model, and maximum step budget matched to the state-query baselines.

A. KnowNo Baseline Implementation

Prompt structure.: At each decision step, KnowNo first generates multiple-choice action options and then scores those options. The GPT-4o and GPT-3.5-turbo variants use the same prompt templates, the same runtime state fields, the same conformal threshold $\hat{q} = 0.92$, and the same scoring temperature 3.0; only the underlying language model is changed. Listings C-A and C-A show the prompt structure used in the experiments. The actual generation prompt includes domain-specific few-shot examples before the runtime state block.

KnowNo tomato-harvesting prompt template

Generation prompt

We: You are a tomato harvesting robot. The robot moves between a dock station and tomato stems, observes tomatoes, picks ripe tomatoes, scans held tomatoes for quality, loads good tomatoes, and discards rotten tomatoes.

Action roles:

- navigate to <location>: move the robot to dock_station, stem_01, or stem_02.
- detect <location>: observe tomatoes at the current robot stem.
- pick <tomato>: pick one detected ripe tomato at the current robot stem with an empty hand.
- scan <tomato>: inspect the currently held tomato.
- place <tomato>: load the held ripe tomato.
- discard <tomato>: discard the held rotten or bad tomato.

Output rules:

- Each option must be exactly one allowed action from the action roles.
- Use only these action formats: navigate to <location>, detect <location>, pick <tomato>, scan, scan <tomato>, place <tomato>, discard <tomato>.
- Do not generate descriptive phrases, retries, requests for assistance, system checks, or any action outside these formats.

<domain-specific few-shot examples for valid option generation>

We: The task is: <instruction>

We: The environment contains four tomatoes: tomato1, tomato2, tomato3, and tomato4. The locations are dock_station, stem_01, and stem_02.

We: The robot is at <robot_location>. <held_tomato_sentence>

We: The tomatoes that still need attention are <active_tomatoes>.

We: <tomato_state_sentences>

We: The loaded tomatoes are <loaded_tomatoes>. The discarded tomatoes are <discarded_tomatoes>.

We: <completed_action_history>

We: What should the robot do next? Answer with four options labeled A), B), C), and D).

You:

Scoring prompt

You are a tomato harvesting robot. The robot moves between a dock station and tomato stems, observes tomatoes, picks ripe tomatoes, scans held tomatoes for quality, loads good tomatoes, and discards rotten tomatoes.

Action roles:

- navigate to <location>: move the robot to dock_station, stem_01, or stem_02.
- detect <location>: observe tomatoes at the current robot stem.
- pick <tomato>: pick one detected ripe tomato at the current robot stem with an empty hand.
- scan <tomato>: inspect the currently held tomato.
- place <tomato>: load the held ripe tomato.
- discard <tomato>: discard the held rotten or bad tomato.

We: The task is: <instruction>

We: The environment contains four tomatoes: tomato1, tomato2, tomato3, and tomato4. The locations are dock_station, stem_01, and stem_02.
 We: The robot is at <robot_location>. <held_tomato_sentence>
 We: The tomatoes that still need attention are <active_tomatoes>.
 We: <tomato_state_sentences>
 We: The loaded tomatoes are <loaded_tomatoes>. The discarded tomatoes are <discarded_tomatoes>.
 We: <completed_action_history>
 We: What should the robot do next?
 You:
 <generated_options>
 We: Which option is correct? Answer with a single capital letter.
 You:

KnowNo waste-sorting prompt template

Generation prompt

We: You are a robot operating in a waste sorting station. You are in front of a counter. There are four bins: a general bin, a plastic bin, a paper bin, and a can bin.

Action roles:

- detect: observe waste attributes for objects on the counter.
- pick <object>: pick one observed object from the counter with an empty hand.
- place <object> into <bin>: place the held object into the bin matching the observed attribute.

Output rules:

- Each option must be exactly one allowed action from the action roles.
- Use only these action formats: detect, pick <object>, place <object> into <bin>.
- Do not generate descriptive phrases, retries, requests for assistance, system checks, or any action outside these formats.

<domain-specific few-shot examples for valid option generation>

We: The task is: <instruction>
 We: The waste objects still on the counter are <remaining_objects>.
 We: The available bins are <available_bins>.
 We: The occluded waste objects are <occluded_objects>.
 We: <observed_waste_sentence> <held_object_sentence>
 We: <completed_action_history>
 We: What should the robot do next? Answer with four options labeled A), B), C), and D).
 You:

Scoring prompt

You are a robot operating in a waste sorting station. You are in front of a counter. There are four bins: a general bin, a plastic bin, a paper bin, and a can bin.

We: The task is: <instruction>
 We: The waste objects still on the counter are <remaining_objects>.
 We: The available bins are <available_bins>.
 We: The occluded waste objects are <occluded_objects>.
 We: <observed_waste_sentence> <held_object_sentence>
 We: <completed_action_history>
 We: What should the robot do next?
 You:
 <generated_options>
 We: Which option is correct? Answer with a single capital letter.
 You:

Prediction set and $\hat{q}$. For each option letter, the implementation extracts the model’s top-token log probability and applies temperature scaling with temperature 3.0 to obtain normalized option scores. The prediction set is then

$$\Gamma_{\hat{q}}(x) = \{y \in \mathcal{Y}(x) \mid \text{score}(y \mid x) \geq 1 - \hat{q}\},$$

where x denotes the current prompt context and $\mathcal{Y}(x)$ is the set of generated option letters. If $\Gamma_{\hat{q}}(x)$ contains a single executable option, KnowNo executes that option. If the set contains multiple options or includes a fallback/ambiguous option, the baseline requests user help at the action level.

Statistically, $\hat{q}$ is the conformal prediction-set threshold. It is computed once from a calibration set by measuring the nonconformity of the correct option,

$$r_i = 1 - \max_{y \in Y_i^{\text{correct}}} \text{score}(y \mid x_i),$$

and taking the finite-sample conformal quantile

$$\hat{q} = \text{Quantile}_{\lceil (n+1)(1-\alpha) \rceil / n} (\{r_i\}_{i=1}^n).$$

Under the usual exchangeability assumption for calibration and test examples, this threshold gives marginal coverage for the

TABLE X: KnowNo model variant comparison. Values are means over logged runs. GPT-4o uses 200 runs per domain. GPT-3.5-turbo uses 200 waste-sorting runs and 69 tomato-harvesting runs available in the experiment logs.

Model	Domain	n	Success (%)	Steps	Queries	Pred. set size	Time (s)
GPT-4o	Waste	200	38.5	7.23	2.06	1.95	23.68
GPT-4o	Tomato	200	42.0	10.73	2.12	2.17	43.17
GPT-4o	All	400	40.2	8.98	2.10	2.06	33.43
GPT-3.5-turbo	Waste	200	34.5	6.42	7.00	3.44	47.55
GPT-3.5-turbo	Tomato	69	2.9	5.14	6.07	3.51	45.24
GPT-3.5-turbo	All	269	26.4	6.09	6.76	3.46	46.96

correct option at the target level. In practical terms, a larger $\hat{q}$ lowers the score threshold $1 - \hat{q}$ and produces larger prediction sets, which increases the chance of asking for help; a smaller $\hat{q}$ produces smaller sets but may omit the correct action more often. In the reported experiments, $\hat{q} = 0.92$ is fixed during execution rather than updated online.

Model variant comparison.: The main policy comparison reports the GPT-4o KnowNo baseline because it was the stronger and more stable action-query variant. We also evaluated the same KnowNo pipeline with GPT-3.5-turbo using the same structured prompt version, $\hat{q} = 0.92$, and scoring temperature 3.0. Table X summarizes the model comparison. GPT-4o achieved higher success with substantially fewer action-level queries and smaller prediction sets. GPT-3.5-turbo produced larger prediction sets and more frequent help requests, and it was especially unstable in tomato harvesting, where many runs terminated after non-executable or invalid action choices. This comparison indicates that the action-query baseline is sensitive to the underlying language model; therefore, the main text uses GPT-4o as the representative KnowNo condition.

APPENDIX D USER STUDY SCENARIOS AND INTERFACES

This appendix summarizes the Wizard-of-Oz user-study materials referenced in Sec. V-C. Each participant completed five waste sorting scenarios and five tomato harvesting scenarios. The mapping between scenarios and interaction conditions was randomized for each participant so that every participant experienced each condition exactly once per domain.

A. Scenario Structure and Questionnaire Timing

Table XI summarizes the scenario identifiers and primary variations used for counterbalancing. The following paragraphs describe the domain-specific scenario structure and post-scenario questionnaires. The scenario IDs in the table are used only to identify the WoZ materials and randomization records; they do not encode the interaction condition assigned to a participant.

TABLE XI: User-study scenario structure. Scenario identifiers were used for counterbalancing and do not imply a fixed condition assignment.

Domain	Scenario IDs	Primary variation
Waste sorting	W1–W5	Waste category, object order, and detection ambiguity
Tomato harvesting	T1–T5	Tomato state, stem location, and scan/harvest ambiguity

1) *Waste Sorting Scenarios*: The waste sorting trials showed a tabletop scene with four waste objects and category-dependent target bins. Participants answered questions about visually highlighted objects or selected actions depending on the assigned condition. In state-level query conditions, the interface asked participants to verify observable task facts such as whether the highlighted object belonged to a specific waste category. In the **KnowNo** condition, participants selected the robot’s next action, which could require deciding whether the robot should inspect an object again or proceed to grasp and place it.

2) *Tomato Harvesting Scenarios*: The tomato harvesting trials showed tomatoes attached to two stems and required participants to reason about whether tomatoes should be harvested, ignored, scanned, placed, or discarded. State-level query conditions asked participants to verify facts such as object existence or tomato condition. In **KnowNo**, participants selected action-level alternatives in the harvesting sequence. These action choices were generally more constrained than in waste sorting because the tomato workflow follows a clearer navigation–detection–pick–scan–place/discard order.

Fig. 11 shows representative interface screens from the user study. The examples include a state-level query interface and action-level KnowNo interfaces to show the difference between verifying a task fact and selecting the next robot action.

3) *Questionnaire Timing*: After each scenario, participants completed NASA Raw-TLX, fatigue, SART, and SAGAT-style items. SAGAT questions were grouped into three levels: Lv1 for perception of the current state, Lv2 for comprehension of the task situation, and Lv3 for projection of future robot behavior. The analyses in Sec. V-C6 and Sec. V-C7 use these post-scenario responses.

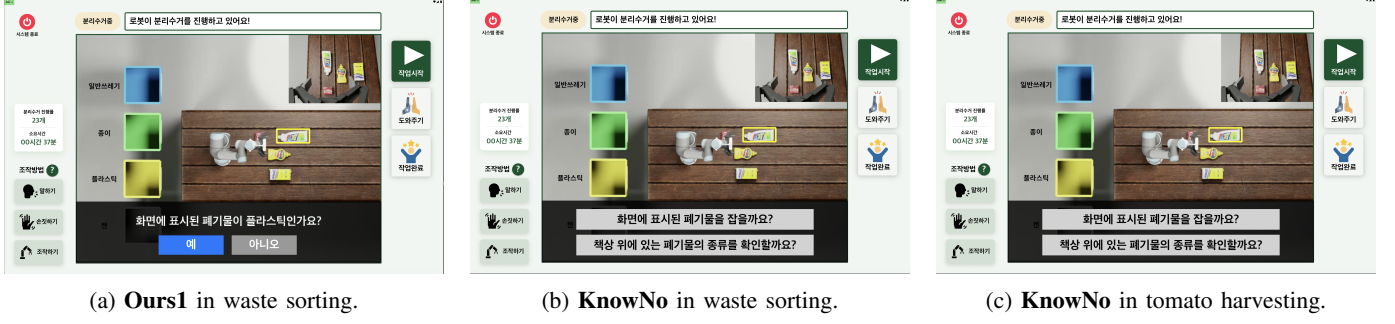


Fig. 11: Representative user-study interfaces. The proposed state-query interface asks users to verify observable task facts, whereas the action-query baseline asks users to choose the next robot action.

B. Condition Codes and Participant-Level Assignment

Table XII defines the condition codes used in the user-study logs. Table XIII reports the participant-level trial order reconstructed from the survey timestamps. Each token has the form $D_S \times C_Y$, where D is the domain ($\bar{W}$: waste sorting, T : tomato harvesting), S_x is the scenario identifier, and C_Y is the condition code. Participants completed the two domain blocks in different orders, so the table is split into the first five and last five trials rather than assuming that tomato harvesting always preceded waste sorting. The condition-code table is used to decode the compact trial-order tokens and the scenario-level answer summaries below.

TABLE XII: Condition-code definitions used in the user-study logs.

Code	Condition	Interaction policy
C1	All	Query whenever feedback is available
C2	No	No user feedback request
C3	Ours1	State-level uncertainty query policy
C4	Ours2	State-level query exposure condition with varied transition profile
C5	KnowNo	Action-level query baseline

C. Questionnaires

The questionnaires were administered in Korean. The following English translation lists the questionnaire content used in the user study. Tables XIV and XV list the SAGAT-style items used after tomato-harvesting and waste-sorting scenarios, respectively. The item levels correspond to the Lv1–Lv3 situation-awareness categories reported in Sec. V-C7. Table XVI lists the post-scenario workload, fatigue, and SART items administered after each trial.

D. Scenario-Level Answer Summary

Table XVII summarizes the scenario-condition scoring records used for task-response correctness. The columns report the number of scored trials, the mean number of correct responses over the mean number of scorable responses, and the corresponding mean accuracy. Condition codes follow Table XII. The **No** condition (C2) is omitted from this summary because it did not request user responses and therefore did not produce scorable task answers.

TABLE XIII: Participant-level condition assignment and timestamp-reconstructed trial order.

Participant	Trials 1–5	Trials 6–10
p001	T_S1C1, T_S4C4, T_S4C5, T_S2C3, T_S3C2	W_S4C5, W_S1C2, W_S5C1, W_S2C3, W_S3C4
p002	T_S4C5, T_S2C3, T_S1C1, T_S3C2, T_S5C4	W_S3C2, W_S4C3, W_S1C5, W_S2C4, W_S5C1
p003	W_S4C4, W_S3C5, W_S1C1, W_S5C3, W_S2C2	T_S4C3, T_S2C1, T_S1C2, T_S5C4, T_S3C5
p004	W_S3C2, W_S4C1, W_S2C4, W_S5C3, W_S1C5	T_S5C1, T_S2C2, T_S1C3, T_S3C5, T_S4C4
p005	W_S1C2, W_S4C5, W_S2C4, W_S3C1, W_S5C3	T_S2C2, T_S5C4, T_S4C1, T_S3C3, T_S1C5
p006	T_S5C2, T_S2C4, T_S3C5, T_S1C1, T_S4C3	W_S2C2, W_S3C3, W_S5C5, W_S1C4, W_S4C1
p007	T_S3C1, T_S5C4, T_S2C5, T_S1C2, T_S4C3	W_S4C5, W_S1C3, W_S2C4, W_S3C1, W_S5C2
p008	W_S3C1, W_S1C4, W_S4C3, W_S2C5, W_S5C2	T_S1C5, T_S5C2, T_S4C1, T_S2C3, T_S3C4
p009	W_S3C4, W_S5C2, W_S1C1, W_S4C5, W_S2C3	T_S4C1, T_S5C3, T_S2C5, T_S1C4, T_S3C2
p010	W_S4C4, W_S2C2, W_S5C5, W_S1C1, W_S4C3	T_S2C1, T_S1C5, T_S5C3, T_S4C4, T_S3C2
p011	W_S2C2, W_S3C3, W_S1C1, W_S2C4, W_S4C5	T_S2C1, T_S1C5, T_S5C2, T_S4C4, T_S3C3
p012	W_S3C3, W_S5C5, W_S4C4, W_S1C1, W_S2C2	T_S4C2, T_S3C3, T_S2C5, T_S5C4, T_S1C1

TABLE XIV: SAGAT-style questionnaire items for tomato harvesting.

Item	Level	Question	Options
Q1	Lv1	How many tomatoes are currently displayed on the screen?	0; 1; 2; I do not know
Q2	Lv1	What was the state of the tomato most recently processed by the robot?	Ripe; unripe; damaged/rotten; I do not know
Q3	Lv2	What is the key information that the user should judge in the current situation?	Whether the target tomato is harvestable; whether the robot should move to the next location; whether there is information that the robot judged incorrectly; I do not know
Q4	Lv2	Why is user confirmation needed in the current situation?	Tomato ripeness judgment is uncertain; tomato damage judgment is uncertain; a harvest target may have been missed; I do not know
Q5	Lv3	If the user takes no action, what is the robot most likely to do?	Remain waiting; continue the task according to the current judgment; terminate the task; I do not know
Q6	Lv3	If the robot selected the wrong tomato and the user does not correct it, what is the most likely outcome?	The wrong tomato may be harvested; the robot will automatically select the correct tomato again; the task will terminate immediately; I do not know

TABLE XV: SAGAT-style questionnaire items for waste sorting.

Item	Level	Question	Options
Q1	Lv1	How many objects are currently on the table in the screen?	0; 1; 2 or more; I do not know
Q2	Lv1	What was the material/category of the waste object most recently processed by the robot?	Plastic; can; general waste; paper
Q3	Lv2	What is the key information that the user should judge in the current situation?	Whether the next object exists; whether the target object can be grasped from the current position; whether the target object was moved to the correct recycling bin; I do not know
Q4	Lv2	Why was user confirmation needed in the previous situation?	Target object's waste category judgment is uncertain; target object is outside the robot's reachable range; target object is contaminated and its recyclability should be judged; I do not know
Q5	Lv3	If the user takes no action, what is the robot most likely to do?	Remain waiting; continue the waste-sorting task according to the current judgment; terminate the task; I do not know
Q6	Lv3	If the robot sorted an object incorrectly and the user does not correct the task, what is the most likely outcome?	Target object may be placed in the wrong bin; the robot will automatically recognize the error and sort it again; the robot will immediately terminate all waste sorting; I do not know

TABLE XVI: Post-scenario workload, fatigue, and situation-awareness questionnaire items.

Item	Measure	Question
N1	NASA Raw-TLX	How mentally demanding was this task?
N2	NASA Raw-TLX	How physically demanding was this task?
N3	NASA Raw-TLX	How hurried or rushed did the task pace feel?
N4	NASA Raw-TLX	How successful did you feel in performing the requested task?
N5	NASA Raw-TLX	How hard did you have to work to achieve task success?
N6	NASA Raw-TLX	How insecure, discouraged, irritated, stressed, or annoyed did you feel during the task?
F	Cognitive fatigue	What was your level of fatigue?
S1	SART	How unstable or changeable was the situation?
S2	SART	How complicated was the situation?
S3	SART	How many variables were changing within the situation?
S4	SART	How alert were you in the current situation?
S5	SART	How focused were you on the current situation?
S6	SART	How divided was your attention in the current situation?
S7	SART	How much spare cognitive capacity did you have in the current situation?
S8	SART	How much information did you obtain about the current situation?
S9	SART	How good was the information you obtained about the current situation?
S10	SART	How familiar were you with the current situation?

TABLE XVII: Scenario-condition answer summaries for tomato harvesting and waste sorting.

Tomato harvesting					Waste sorting				
Scen.	Cond.	n	Correct/Total	Acc.	Scen.	Cond.	n	Correct/Total	Acc.
S1	C1	4	13.0/13.8	0.95	S1	C1	5	11.8/12.0	0.98
S1	C3	1	8.0/8.0	1.00	S1	C3	1	5.0/5.0	1.00
S1	C4	1	7.0/7.0	1.00	S1	C4	2	6.0/6.0	1.00
S1	C5	4	4.0/4.0	1.00	S1	C5	2	2.0/3.0	0.67
S2	C1	3	13.7/14.0	0.98	S2	C3	2	5.0/5.0	1.00
S2	C3	3	6.3/7.0	0.90	S2	C4	5	5.0/5.0	1.00
S2	C4	1	7.0/7.0	1.00	S2	C5	1	2.0/3.0	0.67
S2	C5	3	4.0/4.0	1.00	S3	C1	3	12.3/12.3	1.00
S3	C1	1	14.0/14.0	1.00	S3	C3	3	5.0/5.0	1.00
S3	C3	3	6.7/7.0	0.95	S3	C4	2	5.0/5.0	1.00
S3	C4	1	6.0/6.0	1.00	S3	C5	1	3.0/3.0	1.00
S3	C5	3	3.0/3.0	1.00	S4	C1	2	12.0/12.0	1.00
S4	C1	3	13.3/13.7	0.98	S4	C3	3	4.3/5.0	0.87
S4	C3	3	7.3/8.0	0.92	S4	C4	3	4.0/4.0	1.00
S4	C4	4	6.8/7.0	0.96	S4	C5	5	2.4/4.0	0.60
S4	C5	1	4.0/4.0	1.00	S5	C1	2	11.5/12.0	0.96
S5	C1	1	11.0/13.0	0.85	S5	C3	3	5.0/5.0	1.00
S5	C3	2	7.0/7.0	1.00	S5	C5	3	3.0/4.0	0.75
S5	C4	5	5.6/6.0	0.93					
S5	C5	1	2.0/3.0	0.67					